Checklist we will follow

Video Checklist

1- Generate Mock Data For Creating Dashboard and Transferring this into a Google Sheet
2- Set Rules and Workflow
3- Describing our Idea to Antigravity
4- Plan the Entire Dashboard Designing
5- Generate Website with Static Data
6- Understanding MCP Server and Setting it up (n8n and supabase)
7- Making Site Dynamic with Realtime Data
8- Making some Changes to Website If needed
9- Push the Code to Cloud (Github)
10- Deploying Website (Vercel)
11- Connecting your Domain

MCP Server Code for supabase

```
{
 "mcpServers": {
   "supu": {
     "command": "npx",
     "args": [
       "mcp-remote",
       "https://mcp.supabase.com/mcp"
     ]
   }
 }
}


N8n mcp server format

{
  "mcpServers": {
    "n8n-mcp": {
      "url": "YOUR_N8N_MCP_SERVER_URL",
      "headers": {
        "Authorization": "Bearer YOUR_N8N_API_KEY"
      }
    }
  }
}
```

Mock Ecom Data Prompt

Mock Ecom Data Prompt

Direct Data Sheet [if you are lazy :)] 🟩 Ecom Mock Sales Data

Generate a complete e-commerce sales dataset in CSV format with exactly 3,539 sales records for a premium Indian skincare brand. Follow these specifications precisely:

DATE RANGE:

- End Date: February 11, 2026
- Start Date: December 23, 2025 (50 days total)
- Distribute sales realistically across all days with variation (some days higher, some lower)
- Apply higher sales on weekends
- Apply festive season bumps (late December for Christmas/New Year, late January for Republic Day)

PRODUCT CATALOG (16 SKUs across 5 categories):

Category 1 - Face Care (5 SKUs):

1. Vitamin C Brightening Serum - ₹2,200
2. Hyaluronic Acid Hydrating Cream - ₹1,850
3. Niacinamide Pore Refining Serum - ₹1,950
4. Gentle Foaming Cleanser - ₹1,100
5. Retinol Night Repair Cream - ₹2,500

Category 2 - Body Care (3 SKUs): 6. Shea Butter Body Lotion - ₹1,400 7. Coffee Arabica Body Scrub - ₹1,200 8. Vitamin E Body Oil - ₹1,650

Category 3 - Eye Care (2 SKUs): 9. Caffeine Eye Cream - ₹1,800 10. Peptide Eye Serum - ₹2,100

Category 4 - Masks & Treatments (3 SKUs): 11. Gold Radiance Sheet Mask (Pack of 5) - ₹950 12. Charcoal Detox Clay Mask - ₹1,350 13. Overnight Hydration Sleeping Mask - ₹1,750

Category 5 - Sun Care (3 SKUs): 14. SPF 50 Sunscreen Gel - ₹850 15. Tinted Sunscreen SPF 40 - ₹1,150 16. After Sun Soothing Gel - ₹900

CUSTOMER DATA:

- Generate approximately 2,500-2,800 unique customers
- Use authentic Indian names (mix of male and female names)
- Email format: firstname.lastname@{gmail.com, yahoo.com, outlook.com, hotmail.com}

- Phone numbers: Indian format starting with +91 followed by 10 digits (use realistic mobile numbers starting with 7, 8, 9, 6)
- 20-25% of total orders should be repeat customers (approximately 700-850 repeat orders)
- CRITICAL: Maximum 3 orders per customer across the entire 50-day period
- For repeat customers, space purchases 7-30 days apart
- Repeat customers can buy same product or different products

LOCATION DATA (Indian cities only): Use these major cities with appropriate state and pin codes:

- Mumbai, Maharashtra (400001-400097)
- Delhi, Delhi (110001-110096)
- Bangalore, Karnataka (560001-560099)
- Chennai, Tamil Nadu (600001-600099)
- Hyderabad, Telangana (500001-500099)
- Pune, Maharashtra (411001-411057)
- Kolkata, West Bengal (700001-700156)
- Ahmedabad, Gujarat (380001-380059)
- Jaipur, Rajasthan (302001-302039)
- Surat, Gujarat (395001-395017)
- Lucknow, Uttar Pradesh (226001-226030)
- Chandigarh, Chandigarh (160001-160047)
- Kochi, Kerala (682001-682035)
- Indore, Madhya Pradesh (452001-452020)
- Nagpur, Maharashtra (440001-440035)

CSV COLUMNS (in this exact order):

1. Order_ID (format: ORD followed by 8-digit number, e.g., ORD10001234)
2. Order_Date (format: DD-MM-YYYY)
3. Customer_Name
4. Email
5. Phone
6. Product_SKU (format: SKU followed by 3 digits, e.g., SKU001 to SKU016)
7. Product_Name
8. Category
9. Unit_Price_INR (₹)
10. Quantity (always 1 for this dataset)
11. Total_Amount_INR (₹)
12. City
13. State
14. Pin_Code
15. Payment_Method (mix of: UPI, Credit Card, Debit Card, Net Banking, Cash on Delivery - with UPI being most common ~40%, COD ~25%, Cards ~20%, Net Banking ~15%)
16. Order_Status (95% "Delivered", 3% "In Transit", 2% "Cancelled")
17. Is_Repeat_Customer (Yes/No)

ADDITIONAL REQUIREMENTS:

- All prices must be between ₹800 and ₹2,800 as specified in the product catalog
- Quantity is always 1 (no bulk orders)
- Total_Amount_INR equals Unit_Price_INR (since quantity is 1)
- Order IDs must be sequential and unique
- Ensure realistic distribution: popular products (Vitamin C Serum, Sunscreen) should have more sales
- Email addresses must be consistent for repeat customers
- Generate exactly 3,539 rows of data
- Format as proper CSV with headers
- Use Indian Rupee symbol (₹) or INR notation consistently

OUTPUT FORMAT: Provide the complete dataset as a downloadable CSV file with all 3,539 records properly formatted and ready for analysis.

Code Quality

Code Quality & Architecture Audit - Vibe Coding Projects

🎯 Purpose

This document provides a comprehensive checklist for auditing code quality, architecture, and technical debt in vibe coding projects (AI-generated or rapid development codebases).

📁 1. Folder Structure & Organization

✅ Industry Standard Structure (React/Next.js)

```
project-root/
├── src/
│   ├── app/            # Next.js 13+ App Router
│   ├── components/     # Reusable components
│   │   ├── ui/         # Base UI components (shadcn/ui pattern)
│   │   ├── features/   # Feature-specific components
│   │   └── layouts/    # Layout components
│   ├── lib/            # Utility functions, configurations
│   ├── hooks/          # Custom React hooks
│   ├── services/       # API services, external integrations
│   ├── types/          # TypeScript type definitions
│   ├── constants/      # App-wide constants
│   ├── contexts/       # React Context providers
│   └── styles/         # Global styles, CSS modules
├── public/             # Static assets
├── tests/              # Test files
└── config files        # Root configuration files
```

❌ Common Anti-Patterns to Flag

- [ ] All components in a single folder with no categorization
- [ ] Mixed concerns (API logic in component files)
- [ ] Deeply nested folder structures (>4 levels)
- [ ] Inconsistent naming conventions across folders
- [ ] Configuration files scattered throughout the project
- [ ] Test files not co-located or in dedicated test folder
- [ ] Assets/images in src instead of public

🔍 Audit Checklist

- [ ] **Clear separation of concerns** - Components, services, and utilities are separated
- [ ] **Consistent naming convention** - PascalCase for components, camelCase for utilities
- [ ] **Logical grouping** - Related files are grouped together
- [ ] **Scalable structure** - Easy to locate and add new features
- [ ] **No circular dependencies** - Check with tools like `madge`

🐛 2. Common Bugs & Errors

Memory Leaks

- [ ] **Uncleared intervals/timeouts** - Check for `setInterval`, `setTimeout` without cleanup
- [ ] **Event listeners not removed** - `addEventListener` without `removeEventListener`
- [ ] **Unclosed subscriptions** - WebSocket, EventSource, RxJS subscriptions
- [ ] **Large state objects** - Unnecessarily large objects in React state

Type Safety Issues

- [ ] **Liberal use of `any` type** - TypeScript projects using `any` excessively
- [ ] **Type assertions without validation** - Unsafe `as` casting
- [ ] **Missing null/undefined checks** - Accessing properties without optional chaining
- [ ] **Implicit any** - Variables without type annotations

Async/Promise Issues

- [ ] **Unhandled promise rejections** - Missing `.catch()` or try-catch blocks
- [ ] **Floating promises** - Promises created but not awaited
- [ ] **Race conditions** - Multiple async operations without proper coordination
- [ ] **Callback hell remnants** - Nested callbacks instead of async/await

React-Specific Bugs

- [ ] **Missing dependencies in useEffect** - Incomplete dependency arrays
- [ ] **Direct state mutations** - Modifying state directly instead of using setState
- [ ] **Keys in lists** - Missing or non-unique keys in mapped arrays
- [ ] **Infinite re-render loops** - useEffect without proper dependencies
- [ ] **Props drilling** - Passing props through many levels instead of context/state management

Performance Issues

- [ ] **Unnecessary re-renders** - Missing React.memo, useMemo, useCallback
- [ ] **Large bundle sizes** - No code splitting or lazy loading
- [ ] **Unoptimized images** - Large images without compression or next/image
- [ ] **Blocking operations on main thread** - Heavy computations without Web Workers
- [ ] **No virtualization for large lists** - Rendering thousands of items without react-window/react-virtualized

---

🔄 3. Callback Hell & Async Patterns

❌ Bad Pattern - Callback Hell

```
// ANTI-PATTERN
getData(function(a) {
  getMoreData(a, function(b) {
    getMoreData(b, function(c) {
      getMoreData(c, function(d) {
        getMoreData(d, function(e) {
          // Do something with 'e'
        });
      });
    });
  });
});
```

✅ Good Pattern - Modern Async/Await

```
// RECOMMENDED
async function fetchData() {
  try {
    const a = await getData();
    const b = await getMoreData(a);
    const c = await getMoreData(b);
    const d = await getMoreData(c);
    const e = await getMoreData(d);
    return e;
  } catch (error) {
    handleError(error);
  }
}
```

Audit Checklist

- [ ] **No nested callbacks** - Maximum 1-2 levels of nesting

- [ ] **Async/await usage** - Modern promise handling
- [ ] **Proper error handling** - All async functions have try-catch
- [ ] **Parallel operations** - Use `Promise.all()` when applicable
- [ ] **Error boundaries** - React Error Boundaries for component errors

---

🧩 4. Component Reusability

Component Design Principles

- [ ] **Single Responsibility** - Each component does one thing well
- [ ] **Composable** - Components can be combined to create complex UIs
- [ ] **Props over hardcoding** - Configurable via props, not internal state
- [ ] **No side effects** - Pure components when possible
- [ ] **Documented props** - TypeScript interfaces or PropTypes

❌ Non-Reusable Component (Anti-Pattern)

```
// BAD - Hardcoded, not reusable
function UserCard() {
  return (
    <div className="card">
      <img src="/user1.jpg" alt="User" />
      <h2>John Doe</h2>
      <p>Software Engineer</p>
      <button onClick={() => alert('Clicked')}>View Profile</button>
    </div>
  );
}
```

✅ Reusable Component (Good Pattern)

```
// GOOD - Flexible and reusable
interface CardProps {
  imageSrc: string;
  title: string;
  description: string;
  onAction: () => void;
  actionLabel?: string;
  variant?: 'primary' | 'secondary';
}

function Card({
  imageSrc,
  title,
  description,
```

```
  onAction,
  actionLabel = 'View',
  variant = 'primary'
}: CardProps) {
  return (
    <div className={`card card--${variant}`}>
      <img src={imageSrc} alt={title} />
      <h2>{title}</h2>
      <p>{description}</p>
      <button onClick={onAction}>{actionLabel}</button>
    </div>
  );
}
```

Reusability Audit Checklist

- [ ] **Component composition** - Can components be composed together?
- [ ] **Prop flexibility** - Components accept props for customization
- [ ] **No hardcoded values** - URLs, text, styles are configurable
- [ ] **Render props/children** - Support for flexible content
- [ ] **Controlled vs Uncontrolled** - Appropriate use of controlled components
- [ ] **Separation of logic and presentation** - Custom hooks for logic extraction

---

🎨 5. Styling Standards

CSS Architecture

- [ ] **Consistent methodology** - CSS Modules, Tailwind, Styled Components (choose one)
- [ ] **No inline styles** - Avoid style={{ }} except for dynamic values
- [ ] **CSS variable usage** - Use CSS custom properties for theming
- [ ] **BEM/utility-first naming** - Consistent naming convention
- [ ] **No !important abuse** - Specificity managed properly
- [ ] **Responsive design** - Mobile-first approach with media queries
- [ ] **Dark mode support** - If applicable, proper theme switching

CSS Variables Standard

```
:root {
  /* Color System */
  --color-primary: #3b82f6;
  --color-secondary: #8b5cf6;
  --color-accent: #ec4899;

  /* Neutrals */
```

```
  --color-background: #ffffff;
  --color-foreground: #0a0a0a;
  --color-muted: #f1f5f9;
  --color-border: #e2e8f0;

  /* Typography */
  --font-sans: 'Inter', sans-serif;
  --font-mono: 'Fira Code', monospace;

  /* Spacing Scale */
  --spacing-xs: 0.25rem;
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 1.5rem;
  --spacing-xl: 2rem;

  /* Border Radius */
  --radius-sm: 0.25rem;
  --radius-md: 0.5rem;
  --radius-lg: 1rem;

  /* Shadows */
  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1);
}

[data-theme="dark"] {
  --color-background: #0a0a0a;
  --color-foreground: #fafafa;
  --color-muted: #1e293b;
  --color-border: #334155;
}
```

📱 6. Responsive Design

Breakpoint Standards

```
/* Mobile First Approach */
/* Base styles: Mobile (< 640px) */

@media (min-width: 640px) { /* sm */ }
@media (min-width: 768px) { /* md */ }
@media (min-width: 1024px) { /* lg */ }
@media (min-width: 1280px) { /* xl */ }
```

```
@media (min-width: 1536px) { /* 2xl */ }
```

Tailwind Responsive Classes

```
<div className="
  w-full          // Mobile
  sm:w-1/2        // Small screens
  md:w-1/3        // Medium screens
  lg:w-1/4        // Large screens
  xl:w-1/5        // Extra large screens
">
  Content
</div>
```

Responsive Audit Checklist

- [ ] **Mobile-first CSS** - Base styles for mobile, media queries for larger screens
- [ ] **Flexible layouts** - Flexbox/Grid instead of fixed widths
- [ ] **Responsive images** - Proper srcset or Next.js Image component
- [ ] **Touch-friendly** - Buttons/links have minimum 44x44px touch targets
- [ ] **Viewport meta tag** - Present in HTML head
- [ ] **No horizontal scroll** - On any screen size
- [ ] **Readable font sizes** - Minimum 16px base font size

🧪 7. Testing Standards

Testing Pyramid

```
      /\
     /  \    E2E Tests (10%)
    /____\
   /      \  Integration Tests (20%)
  /__________\
 /            \ Unit Tests (70%)
/______________\
```

Required Test Coverage

- [ ] **Unit tests** - Individual functions and components
- [ ] **Integration tests** - Component interactions
- [ ] **E2E tests** - Critical user flows (login, checkout, etc.)
- [ ] **Accessibility tests** - Automated a11y checks
- [ ] **Coverage threshold** - Minimum 70% code coverage

Test File Patterns

```
src/
├── components/
│   ├── Button/
│   │   ├── Button.tsx
│   │   ├── Button.test.tsx    # Co-located test
│   │   └── Button.stories.tsx # Storybook story
```

🔧 8. Build & Configuration

Required Configuration Files

- [ ] **TypeScript** - tsconfig.json with strict mode
- [ ] **ESLint** - .eslintrc with recommended rules
- [ ] **Prettier** - .prettierrc for consistent formatting
- [ ] **Git** - .gitignore properly configured
- [ ] **Environment** - .env.example for required env vars
- [ ] **Package manager** - Lock file (package-lock.json, yarn.lock, pnpm-lock.yaml)

Build Optimization Checklist

- [ ] **Code splitting** - Dynamic imports for heavy components
- [ ] **Tree shaking** - Proper ES module imports
- [ ] **Minification** - Production builds are minified
- [ ] **Source maps** - Available for debugging production issues
- [ ] **Bundle analysis** - Regular bundle size monitoring

📊 9. State Management

State Management Patterns

- [ ] **Appropriate tool** - Context API, Zustand, Redux Toolkit (based on complexity)
- [ ] **Normalized state** - Avoid nested state structures
- [ ] **Immutable updates** - Never mutate state directly
- [ ] **Derived state** - Computed values not stored in state
- [ ] **Server state separation** - TanStack Query for API data

Anti-Patterns to Avoid

- [ ] **Prop drilling** - More than 3 levels of props passing
- [ ] **Over-using global state** - Local state when possible
- [ ] **Storing computed values** - Use memoization instead

- [ ] **Mixing server and client state** - Use proper data fetching library

---

🚀 10. Performance Metrics

Core Web Vitals Targets

- [ ] **LCP (Largest Contentful Paint)** - < 2.5s
- [ ] **FID (First Input Delay)** - < 100ms
- [ ] **CLS (Cumulative Layout Shift)** - < 0.1
- [ ] **TTFB (Time to First Byte)** - < 600ms
- [ ] **FCP (First Contentful Paint)** - < 1.8s

Performance Audit Tools

- [ ] **Lighthouse** - Regular audits with scores > 90
- [ ] **WebPageTest** - Detailed performance analysis
- [ ] **Chrome DevTools** - Performance profiling
- [ ] **Bundle Analyzer** - Regular bundle size checks

📝 11. Code Quality Metrics

Maintainability Index

- [ ] **Cyclomatic complexity** - Functions with complexity < 10
- [ ] **Lines of code per file** - Files < 300 lines
- [ ] **Function length** - Functions < 50 lines
- [ ] **Parameter count** - Functions with < 5 parameters
- [ ] **Nesting depth** - Maximum 3-4 levels

Code Smell Detection

- [ ] **Duplicated code** - Use SonarQube/ESLint plugins
- [ ] **Dead code** - Unused imports, variables, functions
- [ ] **Magic numbers** - Extract to named constants
- [ ] **Long parameter lists** - Use object parameters
- [ ] **God objects/components** - Break down large components

---

🔍 12. Code Review Checklist

Before Committing

- [ ] **Code compiles** - No TypeScript errors
- [ ] **Tests pass** - All tests green
- [ ] **Linting passes** - No ESLint errors
- [ ] **Formatting** - Prettier applied
- [ ] **No console.logs** - Remove debug statements
- [ ] **No commented code** - Delete or document
- [ ] **Meaningful commits** - Conventional commit messages

Pull Request Standards

- [ ] **Descriptive title** - Clear summary of changes
- [ ] **Description** - What, why, and how
- [ ] **Screenshots** - For UI changes
- [ ] **Breaking changes** - Clearly documented
- [ ] **Size** - PRs < 400 lines of changes
- [ ] **Reviews** - At least 1 approval required

🛠️ 13. Tooling Recommendations

Essential Dev Tools

- **Linting**: ESLint with TypeScript, React, and accessibility plugins
- **Formatting**: Prettier
- **Type Checking**: TypeScript in strict mode
- **Testing**: Jest/Vitest, React Testing Library, Playwright
- **Bundle Analysis**: webpack-bundle-analyzer or Next.js built-in
- **Performance**: Lighthouse CI, Chrome DevTools
- **Dependencies**: Dependabot, npm audit
- **Code Quality**: SonarQube, CodeClimate

✅ Final Audit Summary Template

```
## Code Quality Audit Results

**Project**: [Project Name]
**Date**: [Audit Date]
**Auditor**: [Name]

### Overall Score: [X]/100

#### Category Scores
- Folder Structure: [X]/10
- Bug & Error Handling: [X]/15
```

- Code Reusability: [X]/15
- Styling Standards: [X]/10
- Responsive Design: [X]/10
- Testing Coverage: [X]/10
- Performance: [X]/15
- State Management: [X]/10
- Code Quality: [X]/5

Critical Issues
1. [Issue description]
2. [Issue description]

High Priority Improvements
1. [Improvement description]
2. [Improvement description]

Recommendations
1. [Recommendation]
2. [Recommendation]

Timeline for Fixes
- Critical: Within 1 week
- High Priority: Within 2 weeks
- Medium Priority: Within 1 month

📚 Additional Resources

- **React Best Practices**: https://react.dev/learn/thinking-in-react
- **TypeScript Guidelines**: https://www.typescriptlang.org/docs/handbook/
- **Web Performance**: https://web.dev/vitals/
- **Accessibility**: https://www.w3.org/WAI/WCAG21/quickref/
- **Clean Code**: Robert C. Martin's "Clean Code" principles

Code Quality & Architecture Audit

Code Quality & Architecture Audit - Vibe Coding Projects

🎯 Purpose

This document provides a comprehensive checklist for auditing code quality, architecture, and technical debt in vibe coding projects (AI-generated or rapid development codebases).

📁 1. Folder Structure & Organization

✅ Industry Standard Structure (React/Next.js)

```
project-root/
├── src/
│   ├── app/            # Next.js 13+ App Router
│   ├── components/     # Reusable components
│   │   ├── ui/         # Base UI components (shadcn/ui pattern)
│   │   ├── features/   # Feature-specific components
│   │   └── layouts/    # Layout components
│   ├── lib/            # Utility functions, configurations
│   ├── hooks/          # Custom React hooks
│   ├── services/       # API services, external integrations
│   ├── types/          # TypeScript type definitions
│   ├── constants/      # App-wide constants
│   ├── contexts/       # React Context providers
│   └── styles/         # Global styles, CSS modules
├── public/             # Static assets
├── tests/              # Test files
└── config files        # Root configuration files
```

❌ Common Anti-Patterns to Flag

- [ ] All components in a single folder with no categorization
- [ ] Mixed concerns (API logic in component files)
- [ ] Deeply nested folder structures (>4 levels)
- [ ] Inconsistent naming conventions across folders
- [ ] Configuration files scattered throughout the project
- [ ] Test files not co-located or in dedicated test folder
- [ ] Assets/images in src instead of public

🔍 Audit Checklist

- [ ] **Clear separation of concerns** - Components, services, and utilities are separated
- [ ] **Consistent naming convention** - PascalCase for components, camelCase for utilities
- [ ] **Logical grouping** - Related files are grouped together
- [ ] **Scalable structure** - Easy to locate and add new features
- [ ] **No circular dependencies** - Check with tools like `madge`

🐛 2. Common Bugs & Errors

Memory Leaks

- [ ] **Uncleared intervals/timeouts** - Check for `setInterval`, `setTimeout` without cleanup
- [ ] **Event listeners not removed** - `addEventListener` without `removeEventListener`
- [ ] **Unclosed subscriptions** - WebSocket, EventSource, RxJS subscriptions
- [ ] **Large state objects** - Unnecessarily large objects in React state

Type Safety Issues

- [ ] **Liberal use of `any` type** - TypeScript projects using `any` excessively
- [ ] **Type assertions without validation** - Unsafe `as` casting
- [ ] **Missing null/undefined checks** - Accessing properties without optional chaining
- [ ] **Implicit any** - Variables without type annotations

Async/Promise Issues

- [ ] **Unhandled promise rejections** - Missing `.catch()` or try-catch blocks
- [ ] **Floating promises** - Promises created but not awaited
- [ ] **Race conditions** - Multiple async operations without proper coordination
- [ ] **Callback hell remnants** - Nested callbacks instead of async/await

React-Specific Bugs

- [ ] **Missing dependencies in useEffect** - Incomplete dependency arrays
- [ ] **Direct state mutations** - Modifying state directly instead of using setState
- [ ] **Keys in lists** - Missing or non-unique keys in mapped arrays
- [ ] **Infinite re-render loops** - useEffect without proper dependencies
- [ ] **Props drilling** - Passing props through many levels instead of context/state management

Performance Issues

- [ ] **Unnecessary re-renders** - Missing React.memo, useMemo, useCallback
- [ ] **Large bundle sizes** - No code splitting or lazy loading
- [ ] **Unoptimized images** - Large images without compression or next/image
- [ ] **Blocking operations on main thread** - Heavy computations without Web Workers
- [ ] **No virtualization for large lists** - Rendering thousands of items without react-window/react-virtualized

---

🔄 3. Callback Hell & Async Patterns

❌ Bad Pattern - Callback Hell

```
// ANTI-PATTERN
getData(function(a) {
  getMoreData(a, function(b) {
    getMoreData(b, function(c) {
      getMoreData(c, function(d) {
        getMoreData(d, function(e) {
          // Do something with 'e'
        });
      });
    });
  });
});
```

✅ Good Pattern - Modern Async/Await

```
// RECOMMENDED
async function fetchData() {
  try {
    const a = await getData();
    const b = await getMoreData(a);
    const c = await getMoreData(b);
    const d = await getMoreData(c);
    const e = await getMoreData(d);
    return e;
  } catch (error) {
    handleError(error);
  }
}
```

Audit Checklist

- [ ] **No nested callbacks** - Maximum 1-2 levels of nesting

- [ ] **Async/await usage** - Modern promise handling
- [ ] **Proper error handling** - All async functions have try-catch
- [ ] **Parallel operations** - Use `Promise.all()` when applicable
- [ ] **Error boundaries** - React Error Boundaries for component errors

---

🧩 4. Component Reusability

Component Design Principles

- [ ] **Single Responsibility** - Each component does one thing well
- [ ] **Composable** - Components can be combined to create complex UIs
- [ ] **Props over hardcoding** - Configurable via props, not internal state
- [ ] **No side effects** - Pure components when possible
- [ ] **Documented props** - TypeScript interfaces or PropTypes

❌ Non-Reusable Component (Anti-Pattern)

```
// BAD - Hardcoded, not reusable
function UserCard() {
  return (
    <div className="card">
      <img src="/user1.jpg" alt="User" />
      <h2>John Doe</h2>
      <p>Software Engineer</p>
      <button onClick={() => alert('Clicked')}>View Profile</button>
    </div>
  );
}
```

✅ Reusable Component (Good Pattern)

```
// GOOD - Flexible and reusable
interface CardProps {
  imageSrc: string;
  title: string;
  description: string;
  onAction: () => void;
  actionLabel?: string;
  variant?: 'primary' | 'secondary';
}

function Card({
  imageSrc,
  title,
  description,
```

```
  onAction,
  actionLabel = 'View',
  variant = 'primary'
}: CardProps) {
  return (
    <div className={`card card--${variant}`}>
      <img src={imageSrc} alt={title} />
      <h2>{title}</h2>
      <p>{description}</p>
      <button onClick={onAction}>{actionLabel}</button>
    </div>
  );
}
```

Reusability Audit Checklist

- [ ] **Component composition** - Can components be composed together?
- [ ] **Prop flexibility** - Components accept props for customization
- [ ] **No hardcoded values** - URLs, text, styles are configurable
- [ ] **Render props/children** - Support for flexible content
- [ ] **Controlled vs Uncontrolled** - Appropriate use of controlled components
- [ ] **Separation of logic and presentation** - Custom hooks for logic extraction

---

🎨 5. Styling Standards

CSS Architecture

- [ ] **Consistent methodology** - CSS Modules, Tailwind, Styled Components (choose one)
- [ ] **No inline styles** - Avoid style={{ }} except for dynamic values
- [ ] **CSS variable usage** - Use CSS custom properties for theming
- [ ] **BEM/utility-first naming** - Consistent naming convention
- [ ] **No !important abuse** - Specificity managed properly
- [ ] **Responsive design** - Mobile-first approach with media queries
- [ ] **Dark mode support** - If applicable, proper theme switching

CSS Variables Standard

```
:root {
  /* Color System */
  --color-primary: #3b82f6;
  --color-secondary: #8b5cf6;
  --color-accent: #ec4899;

  /* Neutrals */
```

```
  --color-background: #ffffff;
  --color-foreground: #0a0a0a;
  --color-muted: #f1f5f9;
  --color-border: #e2e8f0;

  /* Typography */
  --font-sans: 'Inter', sans-serif;
  --font-mono: 'Fira Code', monospace;

  /* Spacing Scale */
  --spacing-xs: 0.25rem;
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 1.5rem;
  --spacing-xl: 2rem;

  /* Border Radius */
  --radius-sm: 0.25rem;
  --radius-md: 0.5rem;
  --radius-lg: 1rem;

  /* Shadows */
  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1);
}

[data-theme="dark"] {
  --color-background: #0a0a0a;
  --color-foreground: #fafafa;
  --color-muted: #1e293b;
  --color-border: #334155;
}
```

📱 6. Responsive Design

Breakpoint Standards

```
/* Mobile First Approach */
/* Base styles: Mobile (< 640px) */

@media (min-width: 640px) { /* sm */ }
@media (min-width: 768px) { /* md */ }
@media (min-width: 1024px) { /* lg */ }
@media (min-width: 1280px) { /* xl */ }
```

```
@media (min-width: 1536px) { /* 2xl */ }
```

Tailwind Responsive Classes

```
<div className="
  w-full          // Mobile
  sm:w-1/2        // Small screens
  md:w-1/3        // Medium screens
  lg:w-1/4        // Large screens
  xl:w-1/5        // Extra large screens
">
  Content
</div>
```

Responsive Audit Checklist

- [ ] **Mobile-first CSS** - Base styles for mobile, media queries for larger screens
- [ ] **Flexible layouts** - Flexbox/Grid instead of fixed widths
- [ ] **Responsive images** - Proper srcset or Next.js Image component
- [ ] **Touch-friendly** - Buttons/links have minimum 44x44px touch targets
- [ ] **Viewport meta tag** - Present in HTML head
- [ ] **No horizontal scroll** - On any screen size
- [ ] **Readable font sizes** - Minimum 16px base font size

🧪 7. Testing Standards

Testing Pyramid

```
      /\
     /  \    E2E Tests (10%)
    /____\
   /      \  Integration Tests (20%)
  /__________\
 /            \ Unit Tests (70%)
/______________\
```

Required Test Coverage

- [ ] **Unit tests** - Individual functions and components
- [ ] **Integration tests** - Component interactions
- [ ] **E2E tests** - Critical user flows (login, checkout, etc.)
- [ ] **Accessibility tests** - Automated a11y checks
- [ ] **Coverage threshold** - Minimum 70% code coverage

Test File Patterns

```
src/
├── components/
│   ├── Button/
│   │   ├── Button.tsx
│   │   ├── Button.test.tsx    # Co-located test
│   │   └── Button.stories.tsx # Storybook story
```

🔧 8. Build & Configuration

Required Configuration Files

- [ ] **TypeScript** - tsconfig.json with strict mode
- [ ] **ESLint** - .eslintrc with recommended rules
- [ ] **Prettier** - .prettierrc for consistent formatting
- [ ] **Git** - .gitignore properly configured
- [ ] **Environment** - .env.example for required env vars
- [ ] **Package manager** - Lock file (package-lock.json, yarn.lock, pnpm-lock.yaml)

Build Optimization Checklist

- [ ] **Code splitting** - Dynamic imports for heavy components
- [ ] **Tree shaking** - Proper ES module imports
- [ ] **Minification** - Production builds are minified
- [ ] **Source maps** - Available for debugging production issues
- [ ] **Bundle analysis** - Regular bundle size monitoring

📊 9. State Management

State Management Patterns

- [ ] **Appropriate tool** - Context API, Zustand, Redux Toolkit (based on complexity)
- [ ] **Normalized state** - Avoid nested state structures
- [ ] **Immutable updates** - Never mutate state directly
- [ ] **Derived state** - Computed values not stored in state
- [ ] **Server state separation** - TanStack Query for API data

Anti-Patterns to Avoid

- [ ] **Prop drilling** - More than 3 levels of props passing
- [ ] **Over-using global state** - Local state when possible
- [ ] **Storing computed values** - Use memoization instead

- [ ] **Mixing server and client state** - Use proper data fetching library

---

🚀 10. Performance Metrics

Core Web Vitals Targets

- [ ] **LCP (Largest Contentful Paint)** - < 2.5s
- [ ] **FID (First Input Delay)** - < 100ms
- [ ] **CLS (Cumulative Layout Shift)** - < 0.1
- [ ] **TTFB (Time to First Byte)** - < 600ms
- [ ] **FCP (First Contentful Paint)** - < 1.8s

Performance Audit Tools

- [ ] **Lighthouse** - Regular audits with scores > 90
- [ ] **WebPageTest** - Detailed performance analysis
- [ ] **Chrome DevTools** - Performance profiling
- [ ] **Bundle Analyzer** - Regular bundle size checks

📝 11. Code Quality Metrics

Maintainability Index

- [ ] **Cyclomatic complexity** - Functions with complexity < 10
- [ ] **Lines of code per file** - Files < 300 lines
- [ ] **Function length** - Functions < 50 lines
- [ ] **Parameter count** - Functions with < 5 parameters
- [ ] **Nesting depth** - Maximum 3-4 levels

Code Smell Detection

- [ ] **Duplicated code** - Use SonarQube/ESLint plugins
- [ ] **Dead code** - Unused imports, variables, functions
- [ ] **Magic numbers** - Extract to named constants
- [ ] **Long parameter lists** - Use object parameters
- [ ] **God objects/components** - Break down large components

---

🔍 12. Code Review Checklist

Before Committing

- [ ] **Code compiles** - No TypeScript errors
- [ ] **Tests pass** - All tests green
- [ ] **Linting passes** - No ESLint errors
- [ ] **Formatting** - Prettier applied
- [ ] **No console.logs** - Remove debug statements
- [ ] **No commented code** - Delete or document
- [ ] **Meaningful commits** - Conventional commit messages

Pull Request Standards

- [ ] **Descriptive title** - Clear summary of changes
- [ ] **Description** - What, why, and how
- [ ] **Screenshots** - For UI changes
- [ ] **Breaking changes** - Clearly documented
- [ ] **Size** - PRs < 400 lines of changes
- [ ] **Reviews** - At least 1 approval required

🛠️ 13. Tooling Recommendations

Essential Dev Tools

- **Linting**: ESLint with TypeScript, React, and accessibility plugins
- **Formatting**: Prettier
- **Type Checking**: TypeScript in strict mode
- **Testing**: Jest/Vitest, React Testing Library, Playwright
- **Bundle Analysis**: webpack-bundle-analyzer or Next.js built-in
- **Performance**: Lighthouse CI, Chrome DevTools
- **Dependencies**: Dependabot, npm audit
- **Code Quality**: SonarQube, CodeClimate

✅ Final Audit Summary Template

```
## Code Quality Audit Results

**Project**: [Project Name]
**Date**: [Audit Date]
**Auditor**: [Name]

### Overall Score: [X]/100

#### Category Scores
- Folder Structure: [X]/10
- Bug & Error Handling: [X]/15
```

- Code Reusability: [X]/15
- Styling Standards: [X]/10
- Responsive Design: [X]/10
- Testing Coverage: [X]/10
- Performance: [X]/15
- State Management: [X]/10
- Code Quality: [X]/5

Critical Issues
1. [Issue description]
2. [Issue description]

High Priority Improvements
1. [Improvement description]
2. [Improvement description]

Recommendations
1. [Recommendation]
2. [Recommendation]

Timeline for Fixes
- Critical: Within 1 week
- High Priority: Within 2 weeks
- Medium Priority: Within 1 month

📚 Additional Resources

- **React Best Practices**: https://react.dev/learn/thinking-in-react
- **TypeScript Guidelines**: https://www.typescriptlang.org/docs/handbook/
- **Web Performance**: https://web.dev/vitals/
- **Accessibility**: https://www.w3.org/WAI/WCAG21/quickref/
- **Clean Code**: Robert C. Martin's "Clean Code" principles

Security Audit Guide

Security Audit Guide - Complete Checklist for Web Applications

🎯 Purpose

Comprehensive security audit checklist covering API key protection, environment variables, database security (Supabase RLS & MongoDB), authentication, and deployment security.

🔐 1. Environment Variables & API Key Security

1.1 Environment Variable Best Practices

✅ Correct Setup

```
# .env.local (Next.js) or .env (Node.js)
# Database
DATABASE_URL="postgresql://..."
SUPABASE_URL="https://xxx.supabase.co"
SUPABASE_ANON_KEY="eyJhbGci..."

# API Keys (prefix determines client/server exposure)
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."  # Client-side safe
STRIPE_SECRET_KEY="sk_test_..."                   # Server-side only
OPENAI_API_KEY="sk-..."                           # Server-side only

# Authentication
NEXTAUTH_SECRET="generated-secret-32-chars-minimum"
NEXTAUTH_URL="http://localhost:3000"

# Third-party Services
SENDGRID_API_KEY="SG...."
CLOUDINARY_API_SECRET="..."
```

🔍 Audit Checklist - Environment Files

- [ ] **.env files in .gitignore** - ALL .env files excluded from version control
- [ ] **.env.example exists** - Template file with dummy values for team
- [ ] **No secrets in code** - Zero hardcoded API keys in source files
- [ ] **Naming convention** - Public keys prefixed (NEXT_PUBLIC_, VITE_, REACT_APP_)
- [ ] **Minimum key length** - Secrets are 32+ characters

- [ ] **No .env in public folder** - Environment files never in public/static directories
- [ ] **Key rotation schedule** - API keys rotated every 90 days

❌ **Common Violations to Flag**

```
// CRITICAL VIOLATION - Hardcoded API Key
const apiKey = "sk_live_51Hp...";  // ❌ NEVER DO THIS

// CRITICAL VIOLATION - Key in client-side code
const stripe = Stripe("sk_test_...");  // ❌ Secret key exposed

// VIOLATION - Committing .env file
git add .env  // ❌ Never commit

// VIOLATION - API key in URL parameters
fetch(`https://api.service.com/data?key=${API_KEY}`)  // ❌ Exposed in logs

// CORRECT - Server-side only
// server-side file only
const apiKey = process.env.OPENAI_API_KEY;  // ✅ Safe
```

1.2 Secret Management in Production

Cloud Provider Solutions

Vercel

```
# Add secrets via CLI
vercel env add OPENAI_API_KEY

# Or via dashboard: Project Settings → Environment Variables
```

AWS

```
# Use AWS Secrets Manager
aws secretsmanager create-secret \
  --name MyAppSecret \
  --secret-string '{"key":"value"}'
```

Google Cloud

```
# Use Secret Manager
gcloud secrets create my-secret --data-file=/path/to/secret.txt
```

Azure

```
# Use Key Vault
az keyvault secret set \
  --vault-name myKeyVault \
  --name mySecret \
  --value mySecretValue
```

Self-Hosted Solutions

```
# HashiCorp Vault
npm install node-vault

# Dotenv Vault (Encrypted .env)
npx dotenv-vault new
npx dotenv-vault push
```

1.3 Client vs Server-Side Exposure

Safe to Expose (Client-Side)

✅ Public API keys (with domain restrictions):

- Stripe Publishable Key
- Google Maps API Key (with HTTP referrer restrictions)
- Firebase Config (with proper security rules)
- Supabase Anon Key (with RLS enabled)

NEVER Expose (Server-Side Only)

❌ Secret/Private keys:

- Stripe Secret Key
- OpenAI API Key
- Database credentials
- JWT signing secrets
- OAuth client secrets
- Email service API keys (SendGrid, Mailgun)

1.4 API Key Security Audit

- [ ] **Keys have restrictions** - IP allowlists, domain restrictions, rate limits
- [ ] **Least privilege** - Keys have minimum required permissions
- [ ] **Separate keys per environment** - Dev, staging, production keys are different
- [ ] **Key monitoring** - Usage tracking and anomaly detection enabled
- [ ] **Automated scanning** - Tools like Gitleaks, TruffleHog in CI/CD
- [ ] **Revocation process** - Documented procedure for compromised keys
- [ ] **No keys in client bundles** - Check with `grep -r "sk_" dist/` for exposed secrets

---

🛡️ 2. Supabase Row Level Security (RLS)

2.1 RLS Fundamentals

Why RLS is Critical

- **Database-level security** - Protection even if API is bypassed
- **User isolation** - Users can only access their own data
- **Admin access control** - Granular permissions by role
- **Multi-tenant support** - Secure data separation

2.2 RLS Audit Checklist

Critical Checks

[ ] **RLS enabled on ALL public tables** - No exposed tables without RLS

```
 -- Check for tables without RLS
SELECT tablename
FROM pg_tables
WHERE schemaname = 'public'
AND tablename NOT IN (
  SELECT tablename
  FROM pg_policies
);
```

-
- [ ] **Policies defined for all operations** - SELECT, INSERT, UPDATE, DELETE
- [ ] **No bypassrls roles exposed** - Service role keys never in client code
- [ ] **JWT claims used correctly** - auth.uid() and auth.jwt() in policies
- [ ] **Storage bucket policies** - RLS on storage.objects table

Example RLS Policies

```
-- Enable RLS
ALTER TABLE todos ENABLE ROW LEVEL SECURITY;

-- ✅ GOOD: Users can only see their own todos
CREATE POLICY "Users can view own todos"
ON todos
FOR SELECT
TO authenticated
USING (auth.uid() = user_id);
```

```
-- ✅ GOOD: Users can only insert their own todos
CREATE POLICY "Users can insert own todos"
ON todos
FOR INSERT
TO authenticated
WITH CHECK (auth.uid() = user_id);

-- ✅ GOOD: Users can update their own todos
CREATE POLICY "Users can update own todos"
ON todos
FOR UPDATE
TO authenticated
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);

-- ✅ GOOD: Users can delete their own todos
CREATE POLICY "Users can delete own todos"
ON todos
FOR DELETE
TO authenticated
USING (auth.uid() = user_id);

-- ✅ GOOD: Public read access for published content
CREATE POLICY "Anyone can view published posts"
ON posts
FOR SELECT
TO anon, authenticated
USING (status = 'published');

-- ✅ GOOD: Admin access
CREATE POLICY "Admins can do everything"
ON posts
FOR ALL
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM user_roles
    WHERE user_id = auth.uid()
    AND role = 'admin'
  )
);
```

2.3 Common RLS Vulnerabilities

❌ Anti-Patterns to Flag

```
-- CRITICAL: No RLS enabled
```

```
-- Tables accessible to everyone without policies

-- CRITICAL: Overly permissive policy
CREATE POLICY "Public access to everything"
ON sensitive_data
FOR ALL
TO public  -- ❌ Never use 'public' role
USING (true);

-- CRITICAL: Using user_metadata for authorization
CREATE POLICY "Check user metadata"
ON data
FOR SELECT
USING (
  (auth.jwt() -> 'user_metadata' ->> 'role') = 'admin'
  -- ❌ User can modify their own metadata!
);

-- BETTER: Use app_metadata
CREATE POLICY "Check app metadata"
ON data
FOR SELECT
USING (
  (auth.jwt() -> 'app_metadata' ->> 'role') = 'admin'
  -- ✅ Only admins can modify app_metadata
);
```

2.4 RLS Performance Optimization

```
-- ✅ Add indexes on columns used in policies
CREATE INDEX idx_todos_user_id ON todos(user_id);
CREATE INDEX idx_user_roles_user_id ON user_roles(user_id);

-- ✅ Use security definer functions for complex checks
CREATE OR REPLACE FUNCTION is_admin()
RETURNS boolean
LANGUAGE sql
SECURITY DEFINER
AS $$
  SELECT EXISTS (
    SELECT 1 FROM user_roles
    WHERE user_id = auth.uid()
    AND role = 'admin'
  );
$$;

-- Then use in policy
```

```
CREATE POLICY "Admin access"
ON sensitive_table
FOR ALL
TO authenticated
USING ((SELECT is_admin()));  -- ✅ Cached per query

-- ✅ Optimize join conditions
-- BAD (slow)
auth.uid() IN (
  SELECT user_id FROM team_users
  WHERE team_users.team_id = table.team_id
)

-- GOOD (fast)
team_id IN (
  SELECT team_id FROM team_users
  WHERE user_id = auth.uid()
)
```

2.5 Storage RLS (File Uploads)

```
-- Enable RLS on storage.objects
ALTER TABLE storage.objects ENABLE ROW LEVEL SECURITY;

-- ✅ Users can upload to their own folder
CREATE POLICY "Users can upload own files"
ON storage.objects
FOR INSERT
TO authenticated
WITH CHECK (
  bucket_id = 'avatars'
  AND (storage.foldername(name))[1] = auth.uid()::text
);

-- ✅ Users can view their own files
CREATE POLICY "Users can view own files"
ON storage.objects
FOR SELECT
TO authenticated
USING (
  bucket_id = 'avatars'
  AND (storage.foldername(name))[1] = auth.uid()::text
);

-- ✅ Public bucket for uploaded images
CREATE POLICY "Public images readable"
ON storage.objects
```

```
FOR SELECT
TO public
USING (bucket_id = 'public-images');
```

2.6 Testing RLS Policies

```
// Test as authenticated user
const { data, error } = await supabase
  .from('todos')
  .select('*');
// Should only return current user's todos

// Test as different user (should fail)
const { data: otherData, error: otherError } = await supabase
  .from('todos')
  .select('*')
  .eq('user_id', 'another-user-id');
// Should return empty or error

// Test as admin
const { data: adminData } = await supabase
  .from('todos')
  .select('*');
// Should return all todos if admin policy exists
```

🗄️ 3. MongoDB Security

3.1 MongoDB Security Fundamentals

Authentication & Authorization

```
// ✅ CORRECT: Use connection string with credentials
const uri = process.env.MONGODB_URI;  // From environment variable
const client = new MongoClient(uri, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
  auth: {
    username: process.env.MONGO_USER,
    password: process.env.MONGO_PASSWORD,
  },
  authSource: 'admin'
});
```

3.2 MongoDB Security Audit Checklist

Connection Security

[ ] **TLS/SSL enabled** - All connections encrypted

```
 const options = {
  tls: true,
  tlsCAFile: '/path/to/ca.pem',  // For MongoDB Atlas
};
```

-
- [ ] **IP Allowlist configured** - MongoDB Atlas network access restricted
- [ ] **Connection string in environment** - Never hardcoded
- [ ] **Authentication enabled** - `--auth` flag in production
- [ ] **Strong passwords** - Minimum 16 characters, random

User Roles & Permissions

```
// ✅ CORRECT: Least privilege principle
use myDatabase;

// Create read-only user for application
db.createUser({
  user: "appReadOnly",
  pwd: passwordPrompt(),  // Prompts for password
  roles: [
    { role: "read", db: "myDatabase" }
  ]
});

// Create application user with limited write access
db.createUser({
  user: "appUser",
  pwd: passwordPrompt(),
  roles: [
    {
      role: "readWrite",
      db: "myDatabase",
      // Restrict to specific collections if possible
    }
  ]
});

// Create admin user (use sparingly)
db.createUser({
  user: "dbAdmin",
  pwd: passwordPrompt(),
```

```
  roles: [
    { role: "dbOwner", db: "myDatabase" }
  ]
});
```

Audit Checklist:

- [ ] **Separate users per environment** - Dev, staging, prod have different credentials
- [ ] **No root user in application** - Applications use limited privilege accounts
- [ ] **User roles documented** - Clear mapping of users to roles
- [ ] **Regular credential rotation** - Passwords changed every 90 days

3.3 Schema Validation (MongoDB's Version of Constraints)

```
// ✅ GOOD: Enforce data structure
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email", "created_at", "user_id"],
      properties: {
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$",
          description: "must be a valid email and is required"
        },
        user_id: {
          bsonType: "string",
          description: "user_id is required"
        },
        created_at: {
          bsonType: "date",
          description: "must be a date and is required"
        },
        role: {
          enum: ["user", "admin", "moderator"],
          description: "can only be one of the enum values"
        }
      }
    }
  },
  validationLevel: "strict",  // Reject invalid documents
  validationAction: "error"   // Don't allow inserts that fail validation
});
```

Audit Checklist:

- [ ] **Schema validation enabled** - For all collections storing user data
- [ ] **Required fields enforced** - Critical fields have validation
- [ ] **Data types validated** - Email, phone, etc. have format validation
- [ ] **Enum constraints** - Status fields restricted to valid values

3.4 Field-Level Access Control (MongoDB Enterprise)

```
// Requires MongoDB Enterprise
// Create role with field-level access
db.runCommand({
  createRole: "restrictedReader",
  privileges: [
    {
      resource: { db: "myDatabase", collection: "users" },
      actions: ["find"],
      // Only allow reading specific fields
      fields: ["email", "username", "created_at"]
      // Password hash field not included
    }
  ],
  roles: []
});
```

3.5 Query-Level Security (Application Layer)

```
// ✅ GOOD: Filter by authenticated user
async function getUserTodos(userId) {
  return await db.collection('todos')
    .find({ user_id: userId })  // Always filter by user_id
    .toArray();
}

// ❌ BAD: No filtering
async function getAllTodos() {
  return await db.collection('todos')
    .find({})  // Returns ALL todos, security issue!
    .toArray();
}

// ✅ GOOD: Prevent injection attacks
async function findUser(email) {
  // Use parameterized queries
  return await db.collection('users').findOne({
    email: email  // Mongo driver handles escaping
  });
}

// ❌ BAD: String concatenation (NoSQL injection risk)
```

```
async function findUserBad(email) {
  return await db.collection('users').findOne(
    JSON.parse(`{"email": "${email}"}`)  // Injection possible!
  );
}
```

3.6 MongoDB Injection Prevention

```
// ❌ VULNERABLE to NoSQL injection
app.get('/api/user', async (req, res) => {
  const { username } = req.query;
  const user = await db.collection('users').findOne({
    username: username  // If username is {"$ne": null}, returns first user!
  });
});

// ✅ SAFE: Validate input type
app.get('/api/user', async (req, res) => {
  const { username } = req.query;

  // Ensure username is a string
  if (typeof username !== 'string') {
    return res.status(400).json({ error: 'Invalid username' });
  }

  const user = await db.collection('users').findOne({
    username: username
  });
});

// ✅ SAFE: Use object-safe libraries
import { sanitize } from 'mongo-sanitize';

app.get('/api/user', async (req, res) => {
  const sanitizedUsername = sanitize(req.query.username);
  const user = await db.collection('users').findOne({
    username: sanitizedUsername
  });
});
```

3.7 Encryption at Rest & In Transit

MongoDB Atlas (Recommended)

- ✅ **Encryption at Rest** - Enabled by default on Atlas
- ✅ **TLS/SSL** - Required for all connections

- ✅ **VPC Peering** - For enhanced network security
- ✅ **Private Endpoints** - For AWS/Azure/GCP

Self-Hosted MongoDB

```
# Enable encryption at rest (requires Enterprise)
mongod --enableEncryption \
  --encryptionKeyFile /path/to/key \
  --encryptionCipherMode AES256-CBC

# Enable TLS
mongod --tlsMode requireTLS \
  --tlsCertificateKeyFile /path/to/cert.pem \
  --tlsCAFile /path/to/ca.pem
```

Audit Checklist:

- [ ] **Encryption at rest enabled** - Data encrypted on disk
- [ ] **TLS for all connections** - No unencrypted traffic
- [ ] **Certificate validation** - Proper cert chain verification
- [ ] **Network encryption** - VPN or VPC peering for DB access

3.8 Audit Logging

```
// Enable audit logging (Enterprise feature)
db.adminCommand({
  setParameter: 1,
  auditAuthorizationSuccess: true
});

// Log authentication attempts
// MongoDB logs to system log or JSON file
```

Audit Checklist:

- [ ] **Authentication logs enabled** - Track login attempts
- [ ] **Query logs reviewed** - Monitor for suspicious patterns
- [ ] **Admin action logging** - Track schema changes, user management
- [ ] **Log retention policy** - Logs kept for minimum 90 days

---

🔒 4. General API Security

4.1 Authentication & Authorization

```
// ✅ GOOD: Verify JWT on every protected route
import { verify } from 'jsonwebtoken';
```

```
async function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
    const decoded = verify(token, process.env.JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Invalid token' });
  }
}

// Apply to protected routes
app.get('/api/protected', authMiddleware, async (req, res) => {
  // req.userId is available here
});
```

4.2 Rate Limiting

```
// Prevent brute force and DDoS
import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP',
  standardHeaders: true,
  legacyHeaders: false,
});

app.use('/api/', limiter);

// Stricter limits for sensitive endpoints
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,  // Only 5 login attempts per 15 minutes
});

app.post('/api/login', authLimiter, loginHandler);
```

4.3 Input Validation & Sanitization

```
// ✅ GOOD: Validate all inputs
import { z } from 'zod';

const userSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8).max(100),
  age: z.number().min(18).max(120).optional(),
});

app.post('/api/register', async (req, res) => {
  try {
    const validatedData = userSchema.parse(req.body);
    // Use validatedData (type-safe and validated)
  } catch (error) {
    return res.status(400).json({ error: error.errors });
  }
});

// ✅ GOOD: Sanitize HTML input
import DOMPurify from 'isomorphic-dompurify';

const sanitizedContent = DOMPurify.sanitize(userInput);
```

4.4 CORS Configuration

```
// ✅ GOOD: Specific origin, not wildcard in production
import cors from 'cors';

const corsOptions = {
  origin: process.env.NODE_ENV === 'production'
    ? 'https://yourdomain.com'  // Specific domain
    : 'http://localhost:3000',   // Local dev
  credentials: true,             // Allow cookies
  optionsSuccessStatus: 200
};

app.use(cors(corsOptions));

// ❌ BAD: Wildcard in production
app.use(cors({ origin: '*' }));  // Never do this!
```

4.5 Security Headers

```
// ✅ GOOD: Use Helmet.js
import helmet from 'helmet';

app.use(helmet({
```

```
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", "data:", "https:"],
    },
  },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true
  },
}));

// Next.js: Add to next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          {
            key: 'X-Frame-Options',
            value: 'DENY',
          },
          {
            key: 'X-Content-Type-Options',
            value: 'nosniff',
          },
          {
            key: 'Referrer-Policy',
            value: 'strict-origin-when-cross-origin',
          },
        ],
      },
    ];
  },
};
```

📋 5. Security Audit Summary Checklist

Environment & Secrets

- [ ] No .env files committed to Git

- [ ] .env.example provided for team
- [ ] All secrets in environment variables
- [ ] Public vs private keys properly separated
- [ ] API keys have restrictions (domain, IP, rate limit)
- [ ] Production secrets use cloud provider secret managers
- [ ] Key rotation schedule in place

Supabase (if applicable)

- [ ] RLS enabled on all public schema tables
- [ ] Policies defined for SELECT, INSERT, UPDATE, DELETE
- [ ] Storage buckets have proper RLS policies
- [ ] anon key used client-side (never service key)
- [ ] Indexes on columns used in RLS policies
- [ ] Security definer functions for complex checks
- [ ] No overly permissive policies (USING true)

MongoDB (if applicable)

- [ ] Authentication enabled
- [ ] TLS/SSL for all connections
- [ ] IP allowlist configured
- [ ] User roles follow least privilege
- [ ] Schema validation on collections
- [ ] Input sanitization for NoSQL injection
- [ ] Encryption at rest enabled
- [ ] Audit logging configured

API Security

- [ ] Authentication on all protected endpoints
- [ ] JWT secrets in environment (32+ chars)
- [ ] Rate limiting implemented
- [ ] Input validation with schemas (Zod, Joi)
- [ ] CORS properly configured
- [ ] Security headers (Helmet.js)
- [ ] HTTPS enforced in production
- [ ] Error messages don't leak sensitive info

Code Quality

- [ ] No console.log with sensitive data
- [ ] Error handling doesn't expose stack traces
- [ ] Dependencies regularly updated
- [ ] npm audit run regularly
- [ ] SAST tools in CI/CD (Snyk, SonarQube)
- [ ] Secret scanning enabled (Gitleaks, GitGuardian)

🚨 6. Incident Response Plan

If API Key is Compromised

1. **Immediate Actions** (within 1 hour)
    - [ ] Revoke compromised key in provider dashboard
    - [ ] Generate new key
    - [ ] Update production environment
    - [ ] Deploy updated configuration
    - [ ] Monitor for unauthorized usage
2. **Investigation** (within 24 hours)
    - [ ] Review audit logs for unauthorized access
    - [ ] Check for data breaches
    - [ ] Identify exposure vector (Git commit, log file, etc.)
    - [ ] Assess impact
3. **Remediation**
    - [ ] Fix vulnerability that caused exposure
    - [ ] Update security procedures
    - [ ] Team training if needed
    - [ ] Document incident for future reference

Emergency Contacts

Service	Contact	Revocation Link
Supabase	support@supabase.io	Dashboard → Settings → API
Stripe	support@stripe.com	Dashboard → Developers → API Keys
OpenAI	support@openai.com	Dashboard → API Keys
MongoDB Atlas	support@mongodb.com	Dashboard → Database Access

---

🛠️ 7. Security Tools & Resources

Automated Scanning

```
# Secret scanning
npm install -g gitleaks
gitleaks detect --source . --verbose

# Dependency vulnerabilities
npm audit
```

```
npm audit fix

# SAST scanning
npm install -g snyk
snyk test
```

CI/CD Integration

```
# GitHub Actions example
name: Security Scan

on: [push, pull_request]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Run Gitleaks
        uses: gitleaks/gitleaks-action@v2

      - name: Run npm audit
        run: npm audit

      - name: Run Snyk
        uses: snyk/actions/node@master
        env:
          SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}
```

Resources

- **OWASP Top 10**: https://owasp.org/www-project-top-ten/
- **Security Headers**: https://securityheaders.com/
- **SSL Labs**: https://www.ssllabs.com/ssltest/
- **MongoDB Security Checklist**: https://docs.mongodb.com/manual/administration/security-checklist/
- **Supabase Security**: https://supabase.com/docs/guides/database/postgres/row-level-security

Vibe Coding Workflow Guide

Vibe Coding Workflow Guide - AI-Assisted Development Best Practices

🎯 Purpose

Complete guide for conducting audits on AI-generated or vibe-coded projects, with emphasis on identifying common patterns, fixing issues, and maintaining code quality throughout rapid development cycles.

🤖 1. What is Vibe Coding?

Definition

Vibe Coding (also called AI-assisted coding or prompt-driven development) is using AI tools (ChatGPT, Claude, Copilot, v0.dev, Bolt, Lovable) to rapidly generate code through natural language descriptions.

Common Characteristics

- ✅ **Rapid prototyping** - MVP in hours instead of days
- ⚠️ **Inconsistent patterns** - Each generation may use different approaches
- ⚠️ **Duplicate code** - AI may recreate similar components
- ⚠️ **Over-engineering** - AI tends to be verbose
- ⚠️ **Outdated practices** - AI trained on older code patterns
- ✅ **Good starting point** - Often produces working code quickly

Popular AI Coding Tools (2025)

Tool	Type	Best For	Output Quality
v0.dev (Vercel)	UI Generator	shadcn/ui components	High
Bolt.new	Full-stack	Complete apps in minutes	Medium
Lovable.dev	Full-stack	Production-ready apps	High
Claude Artifacts	Components	React components	Medium-High
GitHub Copilot	IDE Assistant	Code completion	Variable
Cursor	IDE	Full file generation	High

Codeium	IDE Assistant	Free alternative to Copilot	Medium

📋 2. Vibe Coding Audit Workflow

2.1 Initial Assessment (30 minutes)

Step 1: Project Structure Analysis

```
# Generate project tree
tree -I 'node_modules|.next|dist' -L 3 > structure.txt

# Count files by type
find . -type f -name "*.tsx" | wc -l
find . -type f -name "*.ts" | wc -l
find . -type f -name "*.css" | wc -l
```

Questions to Answer:

- [ ] Is there a clear folder structure?
- [ ] Are components organized by feature or type?
- [ ] Are there obvious duplicate files?
- [ ] Is there a tests folder?
- [ ] Is TypeScript used properly?

Step 2: Dependency Analysis

```
# Check package.json
cat package.json | jq '.dependencies'

# Check for outdated packages
npm outdated

# Check for security vulnerabilities
npm audit
```

Red Flags:

- [ ] Unused dependencies (>30% of packages unused)
- [ ] Duplicate packages (e.g., both lodash and lodash-es)
- [ ] Critical security vulnerabilities
- [ ] Very large bundle size
- [ ] Missing peer dependencies

Step 3: Code Quality Quick Scan

```
# Install and run ESLint
npx eslint . --ext .ts,.tsx
```

```
# Check for TypeScript errors
npx tsc --noEmit

# Count TODO/FIXME comments
grep -r "TODO\|FIXME" src/ | wc -l
```

2.2 Deep Dive Categories

2.2.1 Component Reusability Analysis

Run this analysis:

```
// Find duplicate component patterns
// Create a script: analyze-duplicates.js
const fs = require('fs');
const path = require('path');

function findDuplicateComponents(dir) {
  const components = {};

  function walk(directory) {
    const files = fs.readdirSync(directory);

    files.forEach(file => {
      const filepath = path.join(directory, file);
      const stat = fs.statSync(filepath);

      if (stat.isDirectory()) {
        walk(filepath);
      } else if (file.endsWith('.tsx') || file.endsWith('.jsx')) {
        const content = fs.readFileSync(filepath, 'utf8');
        const lines = content.split('\n').length;

        // Find similar sized components
        const name = file.replace(/\.(tsx|jsx)$/, '');
        if (!components[name]) {
          components[name] = [];
        }
        components[name].push({ filepath, lines });
      }
    });
  }

  walk(dir);

  // Find duplicates
```

```
  Object.entries(components).forEach(([name, files]) => {
    if (files.length > 1) {
      console.log(`\nPotential duplicate: ${name}`);
      files.forEach(f => console.log(`  - ${f.filepath} (${f.lines} lines)`));
    }
  });
}

findDuplicateComponents('./src');
```

Audit Checklist:

- [ ] **No duplicate components** - Similar buttons, cards, modals should be unified
- [ ] **Proper prop interfaces** - Components accept configuration via props
- [ ] **Composition over duplication** - Complex components built from smaller ones
- [ ] **Generic naming** - Components named by function, not specific use case

Example Refactor:

```
// ❌ BAD: Three separate button components
function PrimaryButton() { return <button className="bg-blue-500">Click</button> }
function SecondaryButton() { return <button className="bg-gray-500">Click</button> }
function DangerButton() { return <button className="bg-red-500">Click</button> }

// ✅ GOOD: One reusable component
type ButtonVariant = 'primary' | 'secondary' | 'danger';

interface ButtonProps {
  variant?: ButtonVariant;
  children: React.ReactNode;
  onClick?: () => void;
}

function Button({ variant = 'primary', children, onClick }: ButtonProps) {
  const variants = {
    primary: 'bg-blue-500 hover:bg-blue-600',
    secondary: 'bg-gray-500 hover:bg-gray-600',
    danger: 'bg-red-500 hover:bg-red-600',
  };

  return (
    <button
      className={`${variants[variant]} px-4 py-2 rounded text-white transition`}
      onClick={onClick}
    >
      {children}
    </button>
```

```
  );
}
```

2.2.2 State Management Audit

Common Issues in Vibe Coding:

- Over-using useState for everything
- Prop drilling through 5+ levels
- No separation of server and client state
- Global state for local concerns

Audit Script:

```
# Count useState usage
grep -r "useState" src/ | wc -l

# Find prop drilling (components with many props)
grep -r "interface.*Props" src/ -A 20 | grep -E "^\s+\w+:" | wc -l
```

Refactoring Strategy:

```
// ❌ BAD: Prop drilling
function App() {
  const [user, setUser] = useState(null);
  return <Layout user={user} setUser={setUser} />
}

function Layout({ user, setUser }) {
  return <Sidebar user={user} setUser={setUser} />
}

function Sidebar({ user, setUser }) {
  return <UserMenu user={user} setUser={setUser} />
}

// ✅ GOOD: Context for shared state
const UserContext = createContext(null);

function App() {
  const [user, setUser] = useState(null);

  return (
    <UserContext.Provider value={{ user, setUser }}>
      <Layout />
    </UserContext.Provider>
  );
```

```
}

function UserMenu() {
  const { user, setUser } = useContext(UserContext);
  // Use directly, no prop drilling
}
```

Better: Use a state management library for vibe-coded projects:

```
npm install zustand

// store/userStore.ts
import { create } from 'zustand';

interface UserState {
  user: User | null;
  setUser: (user: User | null) => void;
}

export const useUserStore = create<UserState>((set) => ({
  user: null,
  setUser: (user) => set({ user }),
}));

// Usage anywhere
function UserMenu() {
  const { user, setUser } = useUserStore();
  // No prop drilling, no context needed
}
```

2.2.3 Styling Consistency Audit

Check for Styling Anti-Patterns:

```
# Find inline styles
grep -r "style={{" src/ | wc -l

# Find magic numbers in Tailwind classes
grep -r "w-\[.*px\]" src/

# Check for color inconsistencies
grep -r "bg-\|text-" src/ | sed 's/.*\(bg-[a-z]*-[0-9]*\)/\1/' | sort | uniq -c
```

Issues to Flag:

- [ ] **Inconsistent colors** - 10+ different shades of blue

- [ ] **Magic numbers** - Random px values in Tailwind
- [ ] **Mixed methodologies** - Tailwind + styled-components + CSS modules
- [ ] **No CSS variables** - Colors hardcoded everywhere
- [ ] **Inline styles** - Heavy use of style prop

Standardization Strategy:

1. **Define Design Tokens**:

```
/* styles/design-tokens.css */
:root {
  /* Semantic Color System */
  --color-primary-50: #eff6ff;
  --color-primary-100: #dbeafe;
  --color-primary-500: #3b82f6;
  --color-primary-900: #1e3a8a;

  /* Functional Colors */
  --color-background: #ffffff;
  --color-foreground: #0a0a0a;
  --color-border: #e5e7eb;
  --color-success: #10b981;
  --color-error: #ef4444;
  --color-warning: #f59e0b;

  /* Spacing Scale (use instead of arbitrary values) */
  --spacing-xs: 0.25rem;  /* 4px */
  --spacing-sm: 0.5rem;   /* 8px */
  --spacing-md: 1rem;     /* 16px */
  --spacing-lg: 1.5rem;   /* 24px */
  --spacing-xl: 2rem;     /* 32px */
  --spacing-2xl: 3rem;    /* 48px */

  /* Typography */
  --font-size-xs: 0.75rem;
  --font-size-sm: 0.875rem;
  --font-size-base: 1rem;
  --font-size-lg: 1.125rem;
  --font-size-xl: 1.25rem;
}
```

2. **Extend Tailwind Config**:

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
```

```
      colors: {
        primary: {
          50: 'var(--color-primary-50)',
          100: 'var(--color-primary-100)',
          500: 'var(--color-primary-500)',
          900: 'var(--color-primary-900)',
        },
        background: 'var(--color-background)',
        foreground: 'var(--color-foreground)',
      },
      spacing: {
        'xs': 'var(--spacing-xs)',
        'sm': 'var(--spacing-sm)',
        'md': 'var(--spacing-md)',
        'lg': 'var(--spacing-lg)',
        'xl': 'var(--spacing-xl)',
        '2xl': 'var(--spacing-2xl)',
      }
    }
  }
}
```

3. **Create Reusable Style Utilities**:

```
// lib/cn.ts (class name utility)
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// Usage
<div className={cn(
  "base-styles",
  condition && "conditional-styles",
  props.className  // Allow override
)} />
```

2.2.4 API Integration Audit

Common Vibe Coding Mistakes:

- Fetching data in components without caching
- No loading/error states
- API keys exposed in client code

- No request deduplication
- Unnecessary re-fetching

Audit Checklist:

```
# Find fetch calls
grep -r "fetch(" src/

# Find axios calls
grep -r "axios\." src/

# Check for API keys in code
grep -r "api_key\|apiKey\|API_KEY" src/
```

Red Flags:

- [ ] `fetch()` called directly in component bodies
- [ ] No try-catch blocks around API calls
- [ ] API endpoints hardcoded (not in constants)
- [ ] No loading indicators
- [ ] Token/key management in client code

Best Practice Refactor:

```
// ❌ BAD: Direct fetch in component
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(r => r.json())
      .then(setUser);
  }, [userId]);

  return <div>{user?.name}</div>;
}

// ✅ GOOD: Use TanStack Query
import { useQuery } from '@tanstack/react-query';

function UserProfile({ userId }) {
  const { data: user, isLoading, error } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetch(`/api/users/${userId}`).then(r => r.json()),
    staleTime: 5 * 60 * 1000,  // Cache for 5 minutes
  });

  if (isLoading) return <Skeleton />;
```

```
  if (error) return <ErrorMessage error={error} />;

  return <div>{user.name}</div>;
}
```

Create API Client:

```
// lib/api-client.ts
const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL;

class ApiClient {
  private async request<T>(
    endpoint: string,
    options?: RequestInit
  ): Promise<T> {
    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      headers: {
        'Content-Type': 'application/json',
        ...options?.headers,
      },
      ...options,
    });

    if (!response.ok) {
      throw new Error(`API Error: ${response.statusText}`);
    }

    return response.json();
  }

  async get<T>(endpoint: string): Promise<T> {
    return this.request<T>(endpoint);
  }

  async post<T>(endpoint: string, data: unknown): Promise<T> {
    return this.request<T>(endpoint, {
      method: 'POST',
      body: JSON.stringify(data),
    });
  }
}

export const api = new ApiClient();
```

🔧 3. Automated Audit Tools

3.1 Setup Linting & Formatting

```
# ESLint + Prettier + TypeScript
npm install -D eslint prettier eslint-config-prettier
npm install -D @typescript-eslint/parser @typescript-eslint/eslint-plugin
npm install -D eslint-plugin-react eslint-plugin-react-hooks

# Create .eslintrc.json
{
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:react/recommended",
    "plugin:react-hooks/recommended",
    "prettier"
  ],
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint", "react", "react-hooks"],
  "rules": {
    "react/react-in-jsx-scope": "off",
    "@typescript-eslint/no-explicit-any": "error",
    "@typescript-eslint/explicit-function-return-type": "warn"
  }
}

# Create .prettierrc
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 80,
  "tabWidth": 2
}

# Add to package.json scripts
"scripts": {
  "lint": "eslint . --ext .ts,.tsx",
  "lint:fix": "eslint . --ext .ts,.tsx --fix",
  "format": "prettier --write \"**/*.{ts,tsx,json,md}\""
}
```

3.2 Code Quality Analysis

```
# Install SonarQube scanner
npm install -D sonarqube-scanner
```

```
# Create sonar-project.properties
sonar.projectKey=my-vibe-project
sonar.sources=src
sonar.exclusions=**/node_modules/**,**/*.test.ts
sonar.javascript.lcov.reportPaths=coverage/lcov.info

# Run analysis
npx sonar-scanner
```

3.3 Bundle Size Analysis

```
# Next.js built-in analyzer
npm install -D @next/bundle-analyzer

# In next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
});

module.exports = withBundleAnalyzer({
  // your next config
});

# Run analysis
ANALYZE=true npm run build
```

3.4 Accessibility Testing

```
# Install axe-core
npm install -D @axe-core/react

# Add to _app.tsx (development only)
if (process.env.NODE_ENV !== 'production') {
  import('@axe-core/react').then((axe) => {
    axe.default(React, ReactDOM, 1000);
  });
}

# Or use eslint-plugin-jsx-a11y
npm install -D eslint-plugin-jsx-a11y
```

📊 4. Performance Optimization for Vibe-Coded Apps

4.1 Common Performance Issues

Identify Problems:

```
# Check for large components
find src/ -name "*.tsx" -exec wc -l {} \; | sort -rn | head -10

# Find components without React.memo
grep -L "React.memo\|memo(" src/**/*.tsx

# Check for unnecessary re-renders
# Install why-did-you-render
npm install -D @welldone-software/why-did-you-render
```

4.2 Quick Wins

1. Add React.memo to Pure Components

```
// Before
export function ExpensiveComponent({ data }) {
  return <div>{/* complex rendering */}</div>;
}

// After
export const ExpensiveComponent = React.memo(function ExpensiveComponent({ data }) {
  return <div>{/* complex rendering */}</div>;
});
```

2. Lazy Load Routes/Components

```
// Before
import Dashboard from './Dashboard';
import Settings from './Settings';

// After
const Dashboard = lazy(() => import('./Dashboard'));
const Settings = lazy(() => import('./Settings'));

// Usage
<Suspense fallback={<Spinner />}>
  <Dashboard />
</Suspense>
```

3. Optimize Images

```
// Before
<img src="/large-image.jpg" alt="Hero" />
```

```
// After (Next.js)
import Image from 'next/image';

<Image
  src="/large-image.jpg"
  alt="Hero"
  width={800}
  height={600}
  quality={75}
  loading="lazy"
/>
```

4. Use useCallback for Event Handlers

```
// Before
function TodoList({ todos }) {
  return todos.map(todo => (
    <TodoItem
      key={todo.id}
      todo={todo}
      onDelete={() => deleteTodo(todo.id)}  // New function every render!
    />
  ));
}

// After
function TodoList({ todos }) {
  const handleDelete = useCallback((id) => {
    deleteTodo(id);
  }, []);

  return todos.map(todo => (
    <TodoItem
      key={todo.id}
      todo={todo}
      onDelete={handleDelete}
    />
  ));
}
```

🎯 5. Vibe Coding Best Practices

5.1 Prompting Strategies for Better Code

Good Prompts:

✅ "Create a reusable Button component in TypeScript with variants (primary, secondary, danger) using Tailwind CSS and following accessibility best practices"

✅ "Generate a custom hook for fetching user data from /api/user with loading and error states, TypeScript types, and proper cleanup"

✅ "Build a responsive navbar component using shadcn/ui that works on mobile with a hamburger menu"

Bad Prompts:

❌ "Make a button" (too vague)
❌ "Create everything for a todo app" (too broad)
❌ "Fix this code" (no context)

5.2 Iterative Refinement Process

Workflow:

1. Generate initial code with AI
2. Run linter (`npm run lint`)
3. Fix type errors (`npx tsc --noEmit`)
4. Test in browser
5. Refine prompt and regenerate if needed
6. Extract reusable parts
7. Add to component library

5.3 Documentation Generation

```
# Generate TypeScript docs
npm install -D typedoc
npx typedoc --out docs src/

# Generate component docs with Storybook
npx sb init
```

✅ 6. Audit Completion Checklist

Pre-Audit (1 hour)

- [ ] Clone repository
- [ ] Run `npm install`

- [ ] Run `npm run dev` - does it work?
- [ ] Review package.json dependencies
- [ ] Check environment variables needed
- [ ] Run `npm audit` for security issues

Code Quality (2-3 hours)

- [ ] Set up ESLint and run audit
- [ ] Check TypeScript strict mode compliance
- [ ] Identify duplicate components
- [ ] Review state management patterns
- [ ] Check for proper error handling
- [ ] Verify API calls follow best practices

Performance (1-2 hours)

- [ ] Run Lighthouse audit
- [ ] Check bundle size
- [ ] Identify large components
- [ ] Add React.memo where needed
- [ ] Optimize images
- [ ] Add lazy loading

Security (1 hour)

- [ ] Verify no API keys in code
- [ ] Check .env files in .gitignore
- [ ] Review authentication implementation
- [ ] Verify input validation
- [ ] Check CORS configuration

Styling (1 hour)

- [ ] Identify styling inconsistencies
- [ ] Create design token system
- [ ] Standardize spacing/colors
- [ ] Ensure responsive design
- [ ] Check dark mode support

Testing (2 hours)

- [ ] Set up testing framework
- [ ] Write tests for critical paths
- [ ] Add integration tests
- [ ] Test error scenarios

Documentation (1 hour)

- [ ] Create README with setup instructions
- [ ] Document environment variables
- [ ] Add inline code comments
- [ ] Create component documentation
- [ ] Document API endpoints

Final Report

```
# Vibe Coding Audit Report

**Project**: [Name]
**Date**: [Date]
**Auditor**: [Name]

## Executive Summary
[2-3 sentence overview]

## Metrics
- Total Files: X
- TypeScript Coverage: X%
- Test Coverage: X%
- Bundle Size: X MB
- Lighthouse Score: X/100

## Critical Issues (Fix Immediately)
1. [Issue with severity]
2. [Issue with severity]

## High Priority Improvements
1. [Improvement]
2. [Improvement]

## Technical Debt
1. [Debt item]
2. [Debt item]

## Recommendations
1. [Recommendation]
2. [Recommendation]

## Estimated Refactor Time
- Critical: X hours
- High Priority: X hours
- Tech Debt: X hours
```

📚 7. Resources & Tools

AI Coding Tools

- v0.dev: https://v0.dev
- Bolt: https://bolt.new
- Lovable: https://lovable.dev
- Cursor: https://cursor.sh

Code Quality

- ESLint: https://eslint.org
- Prettier: https://prettier.io
- SonarQube: https://www.sonarqube.org

Performance

- Lighthouse: Chrome DevTools
- Bundle Analyzer: https://www.npmjs.com/package/webpack-bundle-analyzer
- React DevTools Profiler

Learning

- React Patterns: https://reactpatterns.com
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/
- Web.dev: https://web.dev

Project Specification

AI Project Requirements - Complete Specification Guide

🎯 Purpose

This document provides a comprehensive checklist to ensure you provide complete requirements when prompting AI coding tools (v0.dev, Bolt, Claude, ChatGPT, Cursor, etc.). Use this as a pre-flight checklist before generating code to avoid missing critical details.

📋 How to Use This Guide

Method 1: Self-Assessment (Recommended)

1. Read through each section below
2. Check off what applies to your project
3. Copy relevant sections into your AI prompt
4. Include all checked items in your initial request

Method 2: AI-Guided Interview

Copy this prompt to AI:

I want to build [YOUR IDEA]. Please interview me using the "AI Project Requirements Guide" to gather complete specifications. Ask me questions from each category one at a time, and compile a complete specification document at the end.

🎨 1. PROJECT OVERVIEW

Basic Information

- [ ] **Project Name/Working Title**: _____
- [ ] **Project Type**:
    - [ ] SaaS Application
    - [ ] E-commerce Store
    - [ ] Portfolio/Landing Page

    - [ ] Dashboard/Admin Panel
    - [ ] Mobile App (React Native)
    - [ ] Blog/Content Site
    - [ ] Social Platform
    - [ ] AI-Powered Tool
    - [ ] Chrome Extension
    - [ ] Other: _____

[ ] **Primary Goal**: What problem does this solve?

Example: "Help freelancers track invoices and payments in one place"

-
- [ ] **Target Users**: Who will use this?
    - [ ] Age range: _____
    - [ ] Tech proficiency level: (Beginner/Intermediate/Advanced)
    - [ ] Geographic location: _____
    - [ ] User persona: _____

👥 2. USER ROLES & PERMISSIONS

Who Can Access Your App?

- [ ] **Public/Anonymous Users**
    - What can they see? _____
    - What can they do? _____
- [ ] **Registered Users**
    - What features unlock after login? _____
    - What can they create/edit/delete? _____
- [ ] **Admin/Moderators**
    - What admin powers needed? _____
    - User management capabilities? _____
- [ ] **Other Roles** (e.g., Editor, Viewer, Manager)
    - Role name: _____
    - Permissions: _____

Questions to Answer:

✓ Can users sign up themselves or invite-only?
✓ Do different users see different data?
✓ Are there team/organization features?

✓ Is there a role hierarchy?
✓ Can users have multiple roles?

🔐 3. AUTHENTICATION & AUTHORIZATION

Login Methods Required

- [ ] Email + Password
- [ ] Magic Link (passwordless)
- [ ] Google OAuth
- [ ] GitHub OAuth
- [ ] Facebook/Twitter/Apple
- [ ] Phone/SMS
- [ ] SSO (Single Sign-On)
- [ ] Multi-factor Authentication (2FA)

User Profile Information Needed

- [ ] Name (First/Last or Full)
- [ ] Email
- [ ] Phone Number
- [ ] Profile Picture
- [ ] Bio/Description
- [ ] Company/Organization
- [ ] Location/Address
- [ ] Date of Birth
- [ ] Custom fields: _____

Password Requirements

- [ ] Minimum length: _____ characters
- [ ] Require special characters: Yes/No
- [ ] Password reset via: Email / SMS / Both
- [ ] Session timeout: _____ minutes

Questions to Answer:

✓ Can users delete their account?
✓ What happens to user data after deletion?
✓ Email verification required?
✓ Remember me functionality needed?
✓ Social login as primary or optional?

🗂️ 4. DATA STRUCTURE & MODELS

Core Entities (Tables/Collections)

For EACH entity, define:

Entity 1: _____ (e.g., "Product", "Post", "Task")
[ ] **Fields needed**:

Example for "Task":
- id (auto-generated)
- title (string, required, max 200 chars)
- description (text, optional)
- status (enum: todo/in-progress/done)
- priority (enum: low/medium/high)
- due_date (date, optional)
- assigned_to (user_id, foreign key)
- created_at (timestamp)
- updated_at (timestamp)

-
- [ ] **Relationships**:
    - One-to-Many: _____
    - Many-to-Many: _____
    - Belongs to: _____
- [ ] **Validation Rules**:
    - Required fields: _____
    - Unique fields: _____
    - Min/Max values: _____
    - Format constraints: _____

Entity 2: _____

(Repeat structure above)

Common Entities to Consider:

- [ ] Users/Accounts
- [ ] Products/Items
- [ ] Orders/Transactions
- [ ] Posts/Articles/Content
- [ ] Comments/Reviews
- [ ] Categories/Tags
- [ ] Notifications
- [ ] Settings/Preferences
- [ ] Files/Uploads

- [ ] Analytics/Logs

Questions to Answer:

✓ Which fields are searchable?
✓ Which fields are sortable?
✓ Which fields are filterable?
✓ What's the expected data volume? (100 records? 1M records?)
✓ Any fields need to be indexed for performance?
✓ Soft delete or hard delete?
✓ Do you need versioning/history?
✓ Multi-language support needed?

✨ 5. FEATURES & FUNCTIONALITY

Core Features (MVP - Minimum Viable Product)

List in priority order:

1. [ ] **Feature**: _____
    - User story: "As a [user], I want to [action] so that [benefit]"
    - Acceptance criteria:
        - [ ] _____
        - [ ] _____
    - Edge cases to handle: _____
2. [ ] **Feature**: _____ (Repeat structure)
3. [ ] **Feature**: _____

CRUD Operations Required

For each entity, specify:

Example: Tasks

- [ ] **Create**:
    - Who can create? (All users / Admins only / etc.)
    - Required fields: _____
    - Where created? (Modal / Dedicated page / Inline)
- [ ] **Read/View**:
    - List view needed? (Table / Cards / List)
    - Detail view needed? (Modal / Dedicated page)
    - Filtering options: _____
    - Sorting options: _____

    - Search functionality: _____
    - Pagination: (Infinite scroll / Page numbers / Load more)
- [ ] **Update/Edit**:
    - Who can edit? _____
    - Inline editing or form? _____
    - Real-time updates needed? _____
- [ ] **Delete**:
    - Who can delete? _____
    - Soft delete (mark as deleted) or hard delete?
    - Confirmation required? _____
    - Bulk delete needed? _____

Search & Filtering

- [ ] **Global search** across all content
- [ ] **Entity-specific search** (e.g., search only products)
- [ ] **Filters needed**:
    - [ ] By date range
    - [ ] By category/tag
    - [ ] By status
    - [ ] By user/author
    - [ ] By price range
    - [ ] Custom filters: _____
- [ ] **Search features**:
    - [ ] Autocomplete/suggestions
    - [ ] Fuzzy search (typo-tolerant)
    - [ ] Advanced search (multiple criteria)
    - [ ] Search history
    - [ ] Saved searches

File Upload & Management

- [ ] **Upload needed for**:

    - [ ] Profile pictures
    - [ ] Documents (PDF, DOCX)
    - [ ] Images (product photos, posts)
    - [ ] Videos
    - [ ] Audio files
    - [ ] Other: _____
- [ ] **Upload constraints**:

    - Max file size: _____ MB
    - Allowed formats: _____
    - Multiple files at once? Yes/No
    - Drag & drop needed? Yes/No
    - Image cropping/editing? Yes/No

- [ ] **Storage location**:
    - [ ] Cloudinary
    - [ ] AWS S3
    - [ ] Supabase Storage
    - [ ] Vercel Blob
    - [ ] Firebase Storage
    - [ ] Other: _____

Notifications & Communications

- [ ] **Notification types**:
    - [ ] In-app notifications (bell icon)
    - [ ] Email notifications
    - [ ] Push notifications (browser)
    - [ ] SMS notifications
    - [ ] Slack/Discord webhooks
- [ ] **When to notify**:
    - [ ] New user registration
    - [ ] Password reset
    - [ ] New message/comment
    - [ ] Task assigned
    - [ ] Status changes
    - [ ] Payment processed
    - [ ] Custom triggers: _____
- [ ] **Notification preferences**:
    - Can users control which notifications they receive? Yes/No
    - Notification frequency: (Instant / Digest / Custom)

Questions to Answer:

✓ Which features are absolutely critical for launch?
✓ Which features can be added later (Phase 2)?
✓ Any automated workflows/triggers needed?
✓ Do you need scheduling/cron jobs?
✓ Bulk operations required? (bulk edit, bulk delete, export)
✓ Import data from CSV/Excel needed?
✓ Export data to CSV/PDF needed?
✓ Print-friendly views needed?
✓ Offline functionality needed?
✓ Real-time collaboration needed?

🎨 6. UI/UX REQUIREMENTS

Design Preferences

- [ ] **Design Style**:
    - [ ] Minimal/Clean
    - [ ] Corporate/Professional
    - [ ] Playful/Colorful
    - [ ] Dark/Cyberpunk
    - [ ] Glassmorphism
    - [ ] Neumorphism
    - [ ] Reference sites: _____
- [ ] **Color Scheme**:
    - Primary color: _____
    - Secondary color: _____
    - Accent color: _____
    - Dark mode support: Yes/No/Optional
- [ ] **Typography**:
    - Font preference: _____
    - Style: (Modern / Classic / Playful)

Layout & Navigation

- [ ] **Header/Navigation**:
    - Logo position: (Left / Center / Right)
    - Navigation items: _____
    - Search bar in header? Yes/No
    - User menu items: _____
    - Sticky header? Yes/No
- [ ] **Sidebar** (if applicable):
    - Always visible or collapsible?
    - Position: (Left / Right)
    - Contents: _____
- [ ] **Footer**:
    - Links needed: _____
    - Newsletter signup? Yes/No
    - Social media links? Yes/No

Responsive Behavior

- [ ] **Mobile-first design**: Yes/No
- [ ] **Breakpoints priority**:
    - [ ] Mobile (< 640px) - Primary
    - [ ] Tablet (768px - 1024px) - Important

    - [ ] Desktop (> 1024px) - Primary
- [ ] **Mobile-specific features**:
    - [ ] Hamburger menu
    - [ ] Bottom navigation bar
    - [ ] Swipe gestures
    - [ ] Pull to refresh

Page Layouts Needed

For EACH page, specify:

Page: _____ (e.g., "Dashboard", "Product List")

- [ ] **Elements/Sections**:
    - [ ] Header/Title
    - [ ] Summary cards/stats
    - [ ] Main content area
    - [ ] Charts/graphs
    - [ ] Action buttons
    - [ ] Filters/search
    - [ ] Sidebar
    - [ ] Pagination
- [ ] **Layout type**:
    - [ ] Grid (columns: ___)
    - [ ] List/Table
    - [ ] Cards
    - [ ] Kanban board
    - [ ] Calendar view
    - [ ] Custom: _____

Interactive Elements

- [ ] **Modals/Dialogs**:
    - For what actions? _____
    - Size: (Small / Medium / Large / Full-screen)
- [ ] **Dropdowns/Select menus**:
    - Single select or multi-select?
    - Searchable dropdowns needed?
- [ ] **Date/Time pickers**:
    - Date only / Time only / Both
    - Range selection needed?
- [ ] **Rich text editor**:

    - Basic formatting (bold, italic, lists)
    - Advanced (tables, images, code blocks)
    - Markdown support?
- [ ] **Drag & drop**:
    - For what? _____
    - Between lists/columns?
- [ ] **Charts/Graphs needed**:
    - [ ] Line charts
    - [ ] Bar/Column charts
    - [ ] Pie/Donut charts
    - [ ] Area charts
    - [ ] Heatmaps
    - [ ] Real-time updating?

Loading & Empty States

- [ ] **Loading indicators**:
    - Spinner style preference: _____
    - Skeleton loaders? Yes/No
    - Progress bars for uploads? Yes/No
- [ ] **Empty states**:
    - What to show when no data? _____
    - Illustration needed? Yes/No
    - Call-to-action in empty state? _____

Error Handling

- [ ] **Error messages**:
    - Toast notifications
    - Inline errors (form fields)
    - Error pages (404, 500)
    - User-friendly error messages
- [ ] **Validation feedback**:
    - Real-time (as user types)
    - On blur (when leaving field)
    - On submit

Questions to Answer:

✓ Any animation preferences? (Subtle / None / Heavy)
✓ Tooltips needed for complex features?
✓ Help/documentation section needed?

✓ Onboarding flow for new users?
✓ Keyboard shortcuts needed?
✓ Accessibility requirements? (WCAG level A/AA/AAA)
✓ Right-to-left (RTL) support needed?
✓ Print stylesheets needed?

🔌 7. THIRD-PARTY INTEGRATIONS

Payment Processing

- [ ] **Payment provider**:
    - [ ] Stripe
    - [ ] PayPal
    - [ ] Square
    - [ ] Razorpay (India)
    - [ ] Other: _____
- [ ] **Payment features**:
    - [ ] One-time payments
    - [ ] Subscriptions (monthly/yearly)
    - [ ] Free trial period: _____ days
    - [ ] Payment methods: (Credit card / Bank transfer / Wallet)
    - [ ] Invoicing needed? Yes/No
    - [ ] Refunds needed? Yes/No

Email Services

- [ ] **Email provider**:
    - [ ] SendGrid
    - [ ] Mailgun
    - [ ] AWS SES
    - [ ] Resend
    - [ ] Other: _____
- [ ] **Email types needed**:
    - [ ] Transactional (password reset, confirmations)
    - [ ] Marketing/Newsletter
    - [ ] Automated drip campaigns
    - [ ] Custom templates needed? Yes/No

Analytics & Tracking

- [ ] **Analytics tools**:
    - [ ] Google Analytics
    - [ ] Mixpanel
    - [ ] Amplitude
    - [ ] PostHog
    - [ ] Plausible (privacy-focused)
    - [ ] Custom analytics
- [ ] **Events to track**:
    - [ ] Page views
    - [ ] User signups
    - [ ] Feature usage
    - [ ] Conversions
    - [ ] Custom events: _____

Maps & Location

- [ ] **Map provider**:
    - [ ] Google Maps
    - [ ] Mapbox
    - [ ] OpenStreetMap
- [ ] **Features needed**:
    - [ ] Display markers/pins
    - [ ] Geocoding (address to coordinates)
    - [ ] Reverse geocoding
    - [ ] Route planning
    - [ ] Distance calculation
    - [ ] Geofencing

AI/ML Services

- [ ] **AI features**:
    - [ ] Text generation (OpenAI/Claude)
    - [ ] Image generation (DALL-E/Midjourney)
    - [ ] Speech-to-text
    - [ ] Text-to-speech
    - [ ] Content moderation
    - [ ] Sentiment analysis
    - [ ] Recommendations engine
    - [ ] Chatbot/Virtual assistant

Social Media

- [ ] **Social login**: (covered in Auth section)
- [ ] **Social sharing**:
    - [ ] Share buttons (Facebook, Twitter, LinkedIn)
    - [ ] Open Graph meta tags

    - [ ] Twitter Cards
- [ ] **Social feeds**:
    - [ ] Embed Twitter timeline
    - [ ] Instagram feed
    - [ ] YouTube videos

Other Integrations

- [ ] **Communication**:
    - [ ] Slack notifications
    - [ ] Discord webhooks
    - [ ] Twilio (SMS)
    - [ ] WhatsApp Business API
- [ ] **Calendar**:
    - [ ] Google Calendar sync
    - [ ] Outlook Calendar
    - [ ] iCal export
- [ ] **CRM/Marketing**:
    - [ ] HubSpot
    - [ ] Mailchimp
    - [ ] Intercom
- [ ] **Developer Tools**:
    - [ ] GitHub API
    - [ ] Jira
    - [ ] Notion
    - [ ] Airtable

Questions to Answer:

✓ Do you already have API keys for these services?
✓ Any rate limits to be aware of?
✓ Fallback behavior if integration fails?
✓ Webhooks needed for real-time updates?
✓ Data sync frequency? (Real-time / Hourly / Daily)

🛠️ 8. TECHNICAL STACK PREFERENCES

Frontend Framework

- [ ] **Preferred framework**:
    - [ ] Next.js (React) - Recommended for most projects

    - [ ] React (Vite)
    - [ ] Vue.js (Nuxt)
    - [ ] Svelte (SvelteKit)
    - [ ] No preference (AI decides)

UI Component Library

- [ ] **Preference**:
    - [ ] shadcn/ui (copy-paste components) - Recommended
    - [ ] Material UI (MUI)
    - [ ] Chakra UI
    - [ ] Ant Design
    - [ ] Custom components from scratch
    - [ ] No preference

Styling Approach

- [ ] **CSS framework**:
    - [ ] Tailwind CSS - Recommended
    - [ ] CSS Modules
    - [ ] Styled Components
    - [ ] Emotion
    - [ ] Plain CSS/SCSS

State Management

- [ ] **Client state**:

    - [ ] React Context + useState
    - [ ] Zustand - Recommended for simplicity
    - [ ] Redux Toolkit
    - [ ] Jotai
    - [ ] Recoil
- [ ] **Server state** (API data):

    - [ ] TanStack Query (React Query) - Recommended
    - [ ] SWR
    - [ ] RTK Query
    - [ ] Apollo Client (GraphQL)

Backend/Database

- [ ] **Backend preference**:

    - [ ] Supabase (Postgres + Auth + Storage) - Easiest
    - [ ] Firebase (NoSQL + Auth)
    - [ ] MongoDB + Custom API
    - [ ] Next.js API Routes + Prisma

    - [ ] Separate backend (Node.js/Express)
    - [ ] Serverless (Vercel/Netlify Functions)
- [ ] **Database type**:
    - [ ] SQL (Postgres, MySQL)
    - [ ] NoSQL (MongoDB, Firestore)
    - [ ] No preference

Hosting/Deployment

- [ ] **Hosting platform**:
    - [ ] Vercel - Best for Next.js
    - [ ] Netlify
    - [ ] AWS
    - [ ] Google Cloud
    - [ ] DigitalOcean
    - [ ] Fly.io
    - [ ] Railway

Form Handling

- [ ] **Form library**:
    - [ ] React Hook Form + Zod - Recommended
    - [ ] Formik + Yup
    - [ ] TanStack Form
    - [ ] Plain React

Testing (if needed)

- [ ] **Testing requirements**:
    - [ ] Unit tests (Jest/Vitest)
    - [ ] Integration tests (React Testing Library)
    - [ ] E2E tests (Playwright/Cypress)
    - [ ] No testing for MVP

Questions to Answer:

✓ TypeScript or JavaScript? (TypeScript strongly recommended)
✓ Any specific libraries you want included?
✓ Any libraries to avoid?
✓ Development environment? (Mac/Windows/Linux)
✓ Team size working on this? (Solo / Small team / Large)
✓ CI/CD requirements?

📱 9. PLATFORM & DEVICE SUPPORT

Devices

- [ ] **Desktop** (Primary / Secondary / Not important)
- [ ] **Tablet** (Primary / Secondary / Not important)
- [ ] **Mobile** (Primary / Secondary / Not important)

Browser Support

- [ ] **Browsers to support**:
  - [ ] Chrome (latest)
  - [ ] Firefox (latest)
  - [ ] Safari (latest)
  - [ ] Edge (latest)
  - [ ] Mobile browsers
  - [ ] IE 11 (not recommended in 2025)

Operating Systems

- [ ] **Desktop OS**:
  - [ ] Windows
  - [ ] macOS
  - [ ] Linux
- [ ] **Mobile OS**:
  - [ ] iOS (version: ___)
  - [ ] Android (version: ___)

Progressive Web App (PWA)

- [ ] Install to home screen: Yes/No
- [ ] Offline functionality: Yes/No
- [ ] Push notifications: Yes/No
- [ ] Background sync: Yes/No

Questions to Answer:

✓ Native mobile app needed (React Native) or web-first?
✓ Any device-specific features? (camera, GPS, biometrics)
✓ Screen reader compatibility required?
✓ Keyboard navigation required?

🚀 10. PERFORMANCE & SCALABILITY

Performance Requirements

- [ ] **Expected traffic**:
    - Concurrent users: _____
    - Monthly active users: _____
    - Peak traffic time: _____
- [ ] **Performance targets**:
    - Page load time: < _____ seconds
    - API response time: < _____ ms
    - Lighthouse score: > _____ /100

Scalability Considerations

- [ ] **Growth expectations**:
    - Users in 6 months: _____
    - Users in 1 year: _____
    - Data volume: _____ GB/TB
- [ ] **Auto-scaling needed**: Yes/No
- [ ] **CDN for assets**: Yes/No
- [ ] **Database replication**: Yes/No
- [ ] **Caching strategy**:
    - [ ] Browser caching
    - [ ] Server-side caching (Redis)
    - [ ] CDN caching

Questions to Answer:

✓ Any legal requirements? (GDPR, CCPA, HIPAA)
✓ Data retention policies?
✓ Backup and disaster recovery needed?
✓ Uptime requirements? (99.9% SLA)
✓ Geographic distribution of users?

💰 11. BUDGET & TIMELINE

Budget Constraints

- [ ] **Development budget**: $_____
- [ ] **Monthly infrastructure costs**: $_____
- [ ] **Service subscriptions budget**:

    - Auth: _____
    - Database: _____
    - Storage: _____
    - APIs: _____

Timeline

- [ ] **Launch date**: _____
- [ ] **MVP deadline**: _____
- [ ] **Milestones**:
    1. _____
    2. _____
    3. _____

Priorities (rank 1-5)

- [ ] Speed to market: _____
- [ ] Feature completeness: _____
- [ ] Code quality: _____
- [ ] Scalability: _____
- [ ] Cost efficiency: _____

✅ 12. FINAL CHECKLIST BEFORE PROMPTING AI

Must-Have Information

- [ ] Clear project goal and target users
- [ ] List of core features (prioritized)
- [ ] Data models with relationships
- [ ] Authentication requirements
- [ ] UI/UX preferences and reference sites
- [ ] Tech stack preferences (or let AI decide)

Nice-to-Have Information

- [ ] Third-party integrations
- [ ] Performance requirements
- [ ] Budget and timeline
- [ ] Scalability needs

Common Forgotten Items ⚠️

- [ ] Error handling preferences
- [ ] Loading states and empty states
- [ ] Email templates and notifications

- [ ] User roles and permissions
- [ ] File upload requirements
- [ ] Search and filtering needs
- [ ] Mobile responsiveness priority
- [ ] Dark mode support
- [ ] Accessibility requirements
- [ ] SEO requirements (meta tags, sitemap)
- [ ] Multi-language support
- [ ] Terms of Service / Privacy Policy pages
- [ ] Admin panel needs
- [ ] Export/Import data functionality

📝 TEMPLATE FOR AI PROMPT

Copy and fill this out:

```
# Project: [YOUR PROJECT NAME]

## Overview
[Describe your project in 2-3 sentences]

Target users: [Who will use this?]
Main problem it solves: [What problem?]

## Core Features (MVP)
1. [Feature 1] - [Brief description]
2. [Feature 2] - [Brief description]
3. [Feature 3] - [Brief description]
(Add more as needed)

## User Roles & Permissions
- [Role 1]: Can [action1], [action2]
- [Role 2]: Can [action1], [action2]

## Authentication
- Login methods: [Email/Password, Google OAuth, etc.]
- User profile fields: [Name, Email, etc.]
- [Any special auth requirements]

## Data Models

### [Entity 1 Name] (e.g., "Task", "Product")
Fields:
- [field_name] (type, required/optional, constraints)
- [field_name] (type, required/optional, constraints)
```

Relationships:
- [Describe relationships to other entities]

[Entity 2 Name]
[Repeat structure]

UI/UX Requirements
- Design style: [Minimal, Corporate, etc.]
- Color scheme: Primary: [color], Secondary: [color]
- Reference sites: [URLs of sites you like]
- Mobile-first: Yes/No
- Dark mode: Yes/No

Key Pages/Views
1. [Page name]: [Description of what's shown]
2. [Page name]: [Description of what's shown]

Third-Party Integrations
- [Service name]: For [purpose]
- [Service name]: For [purpose]

Technical Preferences
- Frontend: [Next.js, React, etc. or "your choice"]
- UI Library: [shadcn/ui, MUI, etc. or "your choice"]
- Styling: [Tailwind CSS or "your choice"]
- Backend/Database: [Supabase, Firebase, etc. or "your choice"]
- Hosting: [Vercel, Netlify, etc. or "your choice"]

Special Requirements
- [Any specific features not mentioned above]
- [Performance requirements]
- [Compliance requirements]

Timeline
- MVP deadline: [Date or "ASAP"]
- Launch date: [Date or "TBD"]

Questions/Uncertainties
- [Things you're not sure about]
- [Areas where you need AI's recommendation]

🎯 EXAMPLE: FILLED OUT SPEC

Project: TaskFlow - Team Task Management

Overview
A collaborative task management tool for small teams (5-20 people) to organize projects, assign tasks, and track progress. Similar to Trello but simpler and more focused on developer teams.

Target users: Software development teams, tech startups
Main problem it solves: Scattered tasks across Slack, email, and sticky notes

Core Features (MVP)
1. Project boards with columns (To Do, In Progress, Done)
2. Task creation with title, description, assignee, due date
3. Drag and drop tasks between columns
4. Real-time updates when team members make changes
5. Comment on tasks
6. File attachments on tasks

User Roles & Permissions
- Owner: Can create projects, invite members, delete everything
- Member: Can create/edit/delete tasks in projects they're part of
- Viewer: Can only view tasks, cannot edit

Authentication
- Login methods: Email/Password + Google OAuth
- User profile fields: Name, Email, Avatar, Timezone
- Email verification required
- Password reset via email

Data Models

User
Fields:
- id (uuid, auto)
- email (string, required, unique)
- name (string, required)
- avatar_url (string, optional)
- timezone (string, default UTC)
- created_at (timestamp)

Project
Fields:
- id (uuid, auto)
- name (string, required, max 100)
- description (text, optional)
- owner_id (uuid, foreign key to users)
- created_at (timestamp)
- updated_at (timestamp)

Relationships:

- One project has many tasks
- One project has many members (many-to-many)

Task
Fields:
- id (uuid, auto)
- title (string, required, max 200)
- description (text, optional)
- status (enum: todo/in_progress/done)
- priority (enum: low/medium/high, default medium)
- assigned_to (uuid, foreign key to users, optional)
- project_id (uuid, foreign key to projects, required)
- due_date (date, optional)
- position (integer, for ordering)
- created_at (timestamp)
- updated_at (timestamp)

Relationships:
- Belongs to one project
- Assigned to one user (optional)
- Has many comments
- Has many attachments

Comment
Fields:
- id (uuid, auto)
- task_id (uuid, foreign key)
- user_id (uuid, foreign key)
- content (text, required)
- created_at (timestamp)

Attachment
Fields:
- id (uuid, auto)
- task_id (uuid, foreign key)
- file_name (string)
- file_url (string)
- file_size (integer, bytes)
- uploaded_by (uuid, foreign key to users)
- created_at (timestamp)

UI/UX Requirements
- Design style: Minimal, clean, similar to Linear.app
- Color scheme: Primary: Blue (#3b82f6), Secondary: Slate gray
- Reference sites:
  - https://linear.app (for minimal design)
  - https://height.app (for task cards)
- Mobile-first: No, desktop is primary (mobile can be basic)

- Dark mode: Yes, with toggle

Key Pages/Views
1. Dashboard: Grid of all projects user has access to
2. Project Board: Kanban view with columns for task statuses
3. Task Detail Modal: Full task info with comments and attachments
4. Project Settings: Invite members, change project name/description

Third-Party Integrations
- Supabase Storage: For file attachments
- SendGrid: For email notifications (task assigned, comments, mentions)
- Google OAuth: For authentication

Technical Preferences
- Frontend: Next.js 14 with App Router
- UI Library: shadcn/ui with Radix primitives
- Styling: Tailwind CSS
- State: TanStack Query for server state, Zustand for client state
- Backend/Database: Supabase (Postgres + Auth + Storage + Realtime)
- Drag & Drop: dnd-kit
- Hosting: Vercel

Special Requirements
- Real-time updates: When someone moves a task, others see it instantly
- File uploads: Max 10MB per file, accept images, PDFs, docs
- Notifications: Email when task assigned or mentioned in comment
- Search: Ability to search tasks by title/description across all projects
- Keyboard shortcuts: N for new task, / for search
- Row Level Security: Users can only see projects they're invited to

Timeline
- MVP deadline: 2 weeks
- Launch date: 1 month (with testing)

Questions/Uncertainties
- Should we add labels/tags to tasks in MVP or later?
- Is a calendar view needed for due dates or can that be phase 2?
- Should there be time tracking on tasks?

🤖 AI Interview Mode Prompt

If you want AI to interview you instead, copy this:

I want to build [YOUR BASIC IDEA]. I need you to act as a product manager and interview me to gather complete requirements. Ask me ONE question at

a time from these categories in order:

1. Project Overview (goal, users, problem)
2. User Roles & Permissions
3. Authentication needs
4. Data Structure (entities and relationships)
5. Core Features (what users can do)
6. UI/UX preferences
7. Third-party integrations
8. Technical stack preferences
9. Performance & scalability needs
10. Timeline & budget

After each answer, ask the next logical question. If my answer is vague, ask follow-up questions for clarity. Once you have all information, create a complete specification document that I can use to generate code with AI tools like v0.dev or Bolt.

Let's start with the first question!

🎓 Common Pitfalls to Avoid

❌ Too Vague

"Build me a social media app"

Problem: No details about features, users, or requirements

❌ Too Specific (Micromanaging)

"Use exactly these npm packages: react-query v3.39.2, axios v0.27.2, styled-components v5.3.6, and implement using the observer pattern with dependency injection..."

Problem: Over-constraining AI, prevents it from using better solutions

✅ Just Right

"Build a Twitter-like microblogging platform where users can post short messages (max 280 chars), follow other users, and like/retweet posts. Use Next.js, Tailwind CSS, and Supabase. I prefer shadcn/ui components."

Perfect: Clear goal, key features, tech preferences, but flexible

---

🏁 Quick Start Checklist

Before prompting AI, ensure you can answer:

1. ✅ **What** is the app? (1 sentence)
2. ✅ **Who** will use it?
3. ✅ **Why** are you building it? (problem it solves)
4. ✅ **What** are the 3-5 core features?
5. ✅ **What** data needs to be stored?
6. ✅ **How** should it look? (1-2 reference sites)
7. ✅ **When** do you need it? (timeline)

If you can answer these 7 questions, you're ready to prompt AI!

📚 Resources

- **Example Projects**: https://v0.dev/chat (see examples)
- **Design Inspiration**: https://dribbble.com, https://mobbin.com
- **Component Libraries**: https://ui.shadcn.com
- **Data Modeling**: https://dbdiagram.io
- **User Stories**: https://www.mountaingoatsoftware.com/agile/user-stories

---

Pro Tip: Save your filled-out spec in a file! You can refine it as you iterate with AI and reuse parts for future projects. 🚀

UI Design Guide

UI Design & Component Library Guide - React Ecosystem

🎯 Purpose

Complete guide for selecting and implementing UI components, design systems, and libraries for React applications with industry-standard patterns.

📦 1. Component Library Categories

1.1 Headless/Unstyled Libraries

Philosophy: Behavior and accessibility without styling - full design control

Library	Best For	Bundle Size	Accessibility	GitHub Stars
Radix UI	Design systems, Tailwind users	~16kb (primitives)	WCAG 2.1 AA	16.9k
Headless UI	Tailwind integration	Lightweight	Built-in	27.2k
React Aria (Adobe)	Complex interactions, accessibility-first	Modular	Industry-leading	13.8k
Ariakit	Flexible APIs, accessibility	~10kb	WCAG 2.1 AA	8k+
Base UI (MUI)	Material Design customization	Modular	Strong	5k+

Radix UI - Quick Start

```
npm install @radix-ui/react-dialog @radix-ui/react-dropdown-menu
# Or install all primitives
npm install @radix-ui/react

// Example: Dialog Component
import * as Dialog from '@radix-ui/react-dialog';

function MyDialog() {
  return (
```

```
    <Dialog.Root>
      <Dialog.Trigger>Open</Dialog.Trigger>
      <Dialog.Portal>
        <Dialog.Overlay className="overlay" />
        <Dialog.Content className="dialog-content">
          <Dialog.Title>Title</Dialog.Title>
          <Dialog.Description>Description</Dialog.Description>
          <Dialog.Close>Close</Dialog.Close>
        </Dialog.Content>
      </Dialog.Portal>
    </Dialog.Root>
  );
}
```

1.2 Copy-Paste Component Systems

Philosophy: Own the code - no npm dependencies, full customization

shadcn/ui ⭐ (Most Popular for 2025)

```
npx shadcn@latest init
npx shadcn@latest add button dialog card
```

Key Features:

- Built on Radix UI + Tailwind CSS
- Copy components into your project
- 50+ components available
- TypeScript by default
- Multiple visual styles (Vega, Nova, Maia, Lyra, Mira)

```
// After installation, components live in your project
import { Button } from "@/components/ui/button"
import { Dialog } from "@/components/ui/dialog"

export function MyComponent() {
  return (
    <Dialog>
      <Button>Click me</Button>
    </Dialog>
  )
}
```

Alternatives:

- **shadcn/create**: New CLI with Base UI support and style variants

- **Shadcn Studio**: Premium component variants and blocks

1.3 Full-Featured Component Libraries

Philosophy: Complete UI packages with styling included

Library	Framework	Best For	Bundle Size	Style System
Material UI (MUI)	React	Enterprise, Google Material Design	~300kb	Emotion/styled
Ant Design	React	Admin panels, data-heavy apps	~500kb	Less/CSS-in-JS
Chakra UI	React	Accessibility, quick prototyping	~200kb	Emotion
Mantine	React	Developer experience, hooks	~150kb	CSS Modules
NextUI	React/Next.js	Modern design, animations	~40kb	Tailwind CSS
PrimeReact	React	Enterprise, complex components	~400kb	Custom CSS

Material UI (MUI) - Quick Start

```
npm install @mui/material @emotion/react @emotion/styled

import Button from '@mui/material/Button';
import { ThemeProvider, createTheme } from '@mui/material/styles';

const theme = createTheme({
  palette: {
    primary: { main: '#1976d2' },
  },
});

function App() {
  return (
    <ThemeProvider theme={theme}>
      <Button variant="contained">Hello World</Button>
    </ThemeProvider>
  );
}
```

📊 2. Data Visualization - Charts & Graphs

2.1 Chart Library Comparison

Library	Rendering	Best For	Bundle Size	Learning Curve	TypeScript
Recharts	SVG	Simple charts, React-native	~100kb	Easy	✅
Chart.js (react-chartjs-2)	Canvas	Quick implementation, animations	~200kb	Easy	✅
Victory	SVG	Accessibility, cross-platform	~150kb	Medium	✅
Nivo	SVG/Canvas/HTML	Beautiful charts, theming	~200kb	Medium	✅
Visx (Airbnb)	SVG	Custom visualizations, performance	~50kb	Hard	✅
Apache ECharts	Canvas	Large datasets, 3D charts	~300kb	Medium	✅
Plotly.js	WebGL	Scientific, 3D, statistical	~3MB	Medium	✅
D3.js	SVG	Full control, complex visualizations	~250kb	Hard	⚠️

2.2 Recharts (Recommended for Most Use Cases)

```
npm install recharts

import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from
'recharts';

const data = [
  { name: 'Jan', value: 400 },
  { name: 'Feb', value: 300 },
  { name: 'Mar', value: 600 },
];
```

```
function MyChart() {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={data}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="name" />
        <YAxis />
        <Tooltip />
        <Line type="monotone" dataKey="value" stroke="#8884d8" />
      </LineChart>
    </ResponsiveContainer>
  );
}
```

2.3 Chart Use Case Matrix

Chart Type	Best Library	Use Case
Line/Bar/Pie (Simple)	**Recharts**	Dashboards, analytics
Real-time Data	**Chart.js**	Live monitoring, streaming data
Large Datasets (10k+ points)	**ECharts**	Financial data, logs
Interactive/Animated	**Nivo**	Data storytelling, presentations
Custom Visualizations	**Visx** or **D3.js**	Unique requirements
3D Charts	**Plotly.js**	Scientific, engineering
Mobile-Friendly	**Victory**	React Native apps
Accessibility Priority	**Victory**	WCAG compliance needed

2.4 Specialized Chart Libraries

```
# Financial/Trading Charts
npm install lightweight-charts  # TradingView charts
npm install react-financial-charts  # Stock charts

# Calendar Heatmaps
npm install react-calendar-heatmap

# Network/Graph Visualization
npm install react-force-graph
npm install cytoscape cytoscape-react

# Treemaps
```

```
npm install recharts  # Has built-in treemap
```

📅 3. Date Pickers & Calendar Components

3.1 Date Picker Comparison

Library	Bundle Size	Features	Framework Support	Accessibility
react-datepicker	~50kb	Full-featured, time picker	React	✅
react-day-picker	~20kb	Lightweight, flexible	React	WCAG 2.1 AA
MUI Date Picker	~100kb	Material Design, i18n	React	✅
NextUI DatePicker	~30kb	Modern, time zones	React/Next.js	✅
Ant Design DatePicker	~80kb	Enterprise, ranges	React	✅
Shadcn Calendar	~15kb	Minimal, customizable	React	✅

3.2 React-DatePicker (Most Popular)

```
npm install react-datepicker
npm install --save-dev @types/react-datepicker  # For TypeScript

import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";

function MyDatePicker() {
  const [startDate, setStartDate] = useState(new Date());

  return (
    <DatePicker
      selected={startDate}
      onChange={(date) => setStartDate(date)}
      showTimeSelect
      dateFormat="MMMM d, yyyy h:mm aa"
    />
  );
}
```

3.3 React-Day-Picker (Lightweight Alternative)

```
npm install react-day-picker date-fns

import { DayPicker } from 'react-day-picker';
import 'react-day-picker/style.css';

function MyCalendar() {
  const [selected, setSelected] = useState<Date>();

  return (
    <DayPicker
      mode="single"
      selected={selected}
      onSelect={setSelected}
      footer={selected ? `Selected: ${selected.toLocaleDateString()}` : 'Pick a day.'}
    />
  );
}
```

3.4 Shadcn Calendar (with Radix + Tailwind)

```
npx shadcn@latest add calendar

import { Calendar } from "@/components/ui/calendar"

export function CalendarDemo() {
  const [date, setDate] = useState<Date | undefined>(new Date())

  return (
    <Calendar
      mode="single"
      selected={date}
      onSelect={setDate}
      className="rounded-md border"
    />
  )
}
```

3.5 Full Calendar Libraries (Event Management)

```
# Google Calendar-like interface
npm install react-big-calendar
npm install moment  # Or date-fns
```

```
# FullCalendar (premium features)
npm install @fullcalendar/react @fullcalendar/daygrid
```

🎯 4. Drag & Drop Libraries

4.1 Drag & Drop Comparison

Library	Best For	Touch Support	Accessibility	Bundle Size	Maintenance
dnd-kit	Modern, performant	✅	Excellent	~10kb	Active ✅
react-beautiful-dnd	Lists (Trello-style)	⚠️ Limited	Excellent	~50kb	Deprecated ❌
hello-pangea/dnd	Lists (fork of rbd)	⚠️ Limited	Excellent	~50kb	Community ⚠️
react-dnd	Complex interactions	✅	Good	~80kb	Active ✅
pragmatic-drag-and-drop	Framework-agnostic	✅	Good	~5kb	Active ✅

4.2 dnd-kit (Recommended)

```
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities

import { DndContext, closestCenter, KeyboardSensor, PointerSensor, useSensor,
useSensors } from '@dnd-kit/core';
import { arrayMove, SortableContext, sortableKeyboardCoordinates, useSortable,
verticalListSortingStrategy } from '@dnd-kit/sortable';
import { CSS } from '@dnd-kit/utilities';

function SortableItem({ id }) {
  const { attributes, listeners, setNodeRef, transform, transition } = useSortable({ id });

  const style = {
    transform: CSS.Transform.toString(transform),
    transition,
  };

  return (
    <div ref={setNodeRef} style={style} {...attributes} {...listeners}>
      {id}
    </div>
```

```
  );
}

function SortableList() {
  const [items, setItems] = useState(['1', '2', '3']);
  const sensors = useSensors(
    useSensor(PointerSensor),
    useSensor(KeyboardSensor, { coordinateGetter: sortableKeyboardCoordinates })
  );

  function handleDragEnd(event) {
    const { active, over } = event;
    if (active.id !== over.id) {
      setItems((items) => {
        const oldIndex = items.indexOf(active.id);
        const newIndex = items.indexOf(over.id);
        return arrayMove(items, oldIndex, newIndex);
      });
    }
  }

  return (
    <DndContext sensors={sensors} collisionDetection={closestCenter}
onDragEnd={handleDragEnd}>
      <SortableContext items={items} strategy={verticalListSortingStrategy}>
        {items.map((id) => <SortableItem key={id} id={id} />)}
      </SortableContext>
    </DndContext>
  );
}
```

4.3 File Upload with Drag & Drop

```
npm install react-dropzone

import { useDropzone } from 'react-dropzone';

function FileUpload() {
  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    onDrop: (files) => console.log(files),
    accept: { 'image/*': [] },
    maxFiles: 5
  });

  return (
    <div {...getRootProps()} className="dropzone">
      <input {...getInputProps()} />
```

```
    {isDragActive ? <p>Drop files here</p> : <p>Drag files here or click to browse</p>}
  </div>
  );
}
```

🎨 5. Animation Libraries

5.1 Animation Library Comparison

Library	Type	Bundle Size	Best For	Learning Curve
Framer Motion	React-specific	~60kb	UI animations, gestures	Easy
React Spring	Physics-based	~40kb	Natural animations	Medium
GSAP	JavaScript	~50kb	Complex sequences	Medium
Auto Animate	Automatic	~3kb	Dead simple animations	Very Easy
React Transition Group	Transitions	~15kb	Enter/exit animations	Easy

5.2 Framer Motion (Recommended)

```
npm install framer-motion

import { motion } from 'framer-motion';

function AnimatedCard() {
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: -20 }}
      whileHover={{ scale: 1.05 }}
      whileTap={{ scale: 0.95 }}
      transition={{ duration: 0.3 }}
    >
      Card Content
    </motion.div>
  );
}
```

5.3 Auto Animate (Zero Config)

```
npm install @formkit/auto-animate

import { useAutoAnimate } from '@formkit/auto-animate/react'

function List() {
  const [parent] = useAutoAnimate()
  const [items, setItems] = useState([1, 2, 3])

  return (
    <ul ref={parent}>
      {items.map(item => <li key={item}>{item}</li>)}
    </ul>
  )
}
```

🎭 6. Form Libraries

6.1 Form Library Comparison

Library	Bundle Size	Validation	Performance	TypeScript
React Hook Form	~9kb	Zod/Yup	Excellent	✅
Formik	~15kb	Yup	Good	✅
TanStack Form	~12kb	Built-in	Excellent	✅
React Final Form	~5kb	Any	Good	✅

6.2 React Hook Form + Zod (Recommended)

```
npm install react-hook-form zod @hookform/resolvers

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});

type FormData = z.infer<typeof schema>;
```

```
function LoginForm() {
  const { register, handleSubmit, formState: { errors } } = useForm<FormData>({
    resolver: zodResolver(schema),
  });

  const onSubmit = (data: FormData) => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('email')} />
      {errors.email && <span>{errors.email.message}</span>}

      <input type="password" {...register('password')} />
      {errors.password && <span>{errors.password.message}</span>}

      <button type="submit">Submit</button>
    </form>
  );
}
```

🖼️ 7. Image & Media Libraries

```
# Image Optimization
npm install next/image  # Next.js built-in

# Image Gallery
npm install yet-another-react-lightbox
npm install react-image-gallery

# Image Cropping
npm install react-easy-crop
npm install react-image-crop

# Lazy Loading
npm install react-lazy-load-image-component

# Icons
npm install lucide-react        # Modern, customizable
npm install react-icons         # Icon pack aggregator
npm install @heroicons/react    # Tailwind's icons
npm install @tabler/icons-react  # Large icon set
```

🎯 8. Table & Data Grid Libraries

8.1 Table Library Comparison

Library	Features	Bundle Size	TypeScript	Virtualization
TanStack Table	Headless, powerful	~15kb	✅	Via plugin
AG Grid	Enterprise features	~200kb	✅	Built-in
MUI Data Grid	Material Design	~100kb	✅	Pro version
React Table (v7)	Legacy, stable	~14kb	✅	Via plugin

8.2 TanStack Table (Recommended)

```
npm install @tanstack/react-table

import { useReactTable, getCoreRowModel, flexRender } from '@tanstack/react-table';

function DataTable({ data, columns }) {
  const table = useReactTable({
    data,
    columns,
    getCoreRowModel: getCoreRowModel(),
  });

  return (
    <table>
      <thead>
        {table.getHeaderGroups().map(headerGroup => (
          <tr key={headerGroup.id}>
            {headerGroup.headers.map(header => (
              <th key={header.id}>
                {flexRender(header.column.columnDef.header, header.getContext())}
              </th>
            ))}
          </tr>
        ))}
      </thead>
      <tbody>
        {table.getRowModel().rows.map(row => (
          <tr key={row.id}>
            {row.getVisibleCells().map(cell => (
              <td key={cell.id}>
                {flexRender(cell.column.columnDef.cell, cell.getContext())}
              </td>
```

```
          ))}
        </tr>
      ))}
    </tbody>
  </table>
  );
}
```

🔔 9. Notification & Toast Libraries

```
# React Hot Toast (Recommended)
npm install react-hot-toast

# Sonner (Shadcn Alternative)
npm install sonner

# React Toastify
npm install react-toastify

// React Hot Toast
import toast, { Toaster } from 'react-hot-toast';

function App() {
  return (
    <>
      <button onClick={() => toast.success('Success!')}>
        Show Toast
      </button>
      <Toaster position="top-right" />
    </>
  );
}
```

🎨 10. CSS Frameworks & Styling Solutions

10.1 Utility-First CSS

Tailwind CSS (Industry Standard)

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Configuration with CSS Variables:

```
/* globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 0 0% 100%;
    --foreground: 222.2 84% 4.9%;
    --primary: 221.2 83.2% 53.3%;
    --primary-foreground: 210 40% 98%;
  }

  .dark {
    --background: 222.2 84% 4.9%;
    --foreground: 210 40% 98%;
    --primary: 217.2 91.2% 59.8%;
  }
}

@layer base {
  * {
    @apply border-border;
  }
  body {
    @apply bg-background text-foreground;
  }
}
```

10.2 CSS-in-JS Solutions

Library	Performance	Bundle Impact	TypeScript	Server Components
Tailwind CSS	Excellent	No runtime	✅	✅
CSS Modules	Excellent	No runtime	✅	✅
Styled Components	Good	Runtime CSS	✅	⚠️
Emotion	Good	Runtime CSS	✅	⚠️
Vanilla Extract	Excellent	Zero runtime	✅	✅
Panda CSS	Excellent	Zero runtime	✅	✅

🎯 11. Design Patterns & Best Practices

11.1 Component Composition Pattern

```
// Compound Components Pattern
function Card({ children }) {
  return <div className="card">{children}</div>;
}

Card.Header = function CardHeader({ children }) {
  return <div className="card-header">{children}</div>;
};

Card.Body = function CardBody({ children }) {
  return <div className="card-body">{children}</div>;
};

Card.Footer = function CardFooter({ children }) {
  return <div className="card-footer">{children}</div>;
};

// Usage
<Card>
  <Card.Header>Title</Card.Header>
  <Card.Body>Content</Card.Body>
  <Card.Footer>Actions</Card.Footer>
</Card>
```

11.2 Render Props Pattern

```
function DataProvider({ render, endpoint }) {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(endpoint).then(r => r.json()).then(setData);
  }, [endpoint]);

  return render(data);
}

// Usage
<DataProvider
  endpoint="/api/users"
  render={(data) => <UserList users={data} />}
/>
```

11.3 Custom Hook Pattern

```
function useApi(endpoint) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch(endpoint)
      .then(r => r.json())
      .then(setData)
      .catch(setError)
      .finally(() => setLoading(false));
  }, [endpoint]);

  return { data, loading, error };
}

// Usage
function UserList() {
  const { data, loading, error } = useApi('/api/users');

  if (loading) return <Spinner />;
  if (error) return <Error message={error.message} />;
  return <List items={data} />;
}
```

🏗️ 12. Recommended Stacks by Project Type

SaaS Dashboard

```
# UI Framework
shadcn/ui + Radix UI + Tailwind CSS

# Charts
Recharts or Nivo

# Tables
TanStack Table

# Forms
React Hook Form + Zod

# State
Zustand or TanStack Query
```

Notifications
Sonner or React Hot Toast

E-Commerce

UI
NextUI or MUI + Tailwind

Product Gallery
yet-another-react-lightbox

Forms
React Hook Form + Zod

Payments
@stripe/react-stripe-js

Animations
Framer Motion

Admin Panel

UI
Ant Design or MUI

Charts
ECharts for complex visualizations

Tables
AG Grid Community

Forms
Formik or React Hook Form

State
Redux Toolkit

Portfolio/Blog

UI
Shadcn/ui + Tailwind

Animations
Framer Motion

Forms

React Hook Form

CMS
Next.js + Contentlayer or MDX

📚 13. Additional Utility Libraries

State Management

```
npm install zustand             # Lightweight state
npm install @tanstack/react-query  # Server state
npm install jotai               # Atomic state
npm install valtio              # Proxy-based state
```

Utilities

```
npm install clsx                # Conditional classNames
npm install class-variance-authority  # CVA for variants
npm install tailwind-merge      # Merge Tailwind classes
npm install date-fns            # Date manipulation
npm install lodash-es           # Utility functions
```

Performance

```
npm install react-window        # Virtualization
npm install react-virtualized   # Alternative virtualization
npm install use-debounce        # Debouncing
```

✅ Quick Reference Checklist

Before Starting a Project

- [ ] Choose UI framework (shadcn/ui recommended for 2025)
- [ ] Set up Tailwind CSS with proper configuration
- [ ] Install form library (React Hook Form + Zod)
- [ ] Set up state management (Zustand for client, TanStack Query for server)
- [ ] Configure TypeScript in strict mode
- [ ] Set up ESLint + Prettier
- [ ] Choose chart library based on needs
- [ ] Plan responsive breakpoints
- [ ] Set up CSS variables for theming

During Development

- [ ] Follow component composition patterns
- [ ] Create reusable components in `/components/ui`
- [ ] Use CSS variables for all colors and spacing
- [ ] Implement proper loading and error states
- [ ] Add proper TypeScript types
- [ ] Test responsive design on multiple breakpoints
- [ ] Optimize images with next/image or similar
- [ ] Implement proper accessibility (ARIA labels, keyboard navigation)

🔗 Official Documentation Links

- **Radix UI**: https://www.radix-ui.com/
- **shadcn/ui**: https://ui.shadcn.com/
- **Tailwind CSS**: https://tailwindcss.com/
- **Recharts**: https://recharts.org/
- **dnd-kit**: https://dndkit.com/
- **React Hook Form**: https://react-hook-form.com/
- **Framer Motion**: https://www.framer.com/motion/
- **TanStack Table**: https://tanstack.com/table/